Министерство образования, науки и молодежи Республики Крым
Государственное бюджетное профессиональное образовательное учреждение
Республики Крым «Феодосийский политехнический техникум»

Цикловая комиссия информационных технологий

ОТЧЕТ
ПО УЧЕБНОЙ ПРАКТИКЕ УП.11.01
ПО ПМ.11 «РАЗРАБОТКА, АДМИНИСТИРОВАНИЕ И ЗАЩИТА БАЗ ДАННЫХ»

Студентаки 3 курса группы ИСПП 23 2/9
Специальность 09.02.07 Информационные системы и программирование
(квалификация: программист)
Новикова М.А.

(фамилия и инициалы)

Руководитель: Горшкова Д.Е.

Оценка: _____

Феодосия, 2026

РЕФЕРАТ

Новикова М.А. Разработка программного модуля «Школа: учителя, предметы, ученики, журнал успеваемости».

Пояснительная записка по УП.11.01 по ПМ 01.01 Разработка, администрирование и защита баз данных. ГБ ПОУ РК «ФПТ». Группа ИСПП 23 2/9. 2025 г.

Пояснительная записка	53 листа.
Таблицы	6 таблиц.
Иллюстрации	32 иллюстрации.
Ссылки	20 ссылок.
Демонстрационная часть	15 слайдов презентации.

В рамках прохождения учебной практики УП.11.01 по ПМ.11 «Разработка, адмистрирование и защита баз данных» было выполнено индивидуальное задание.

На первом этапе была спроектирована база данных с использованием метода ER-диаграмм. Определена структура и типы данных. На втором этапе была создана БД, таблицы и свойства полей. Заполнена созданная база данных. На третьем этапе разработан интерфейс БД, созданы формы для ввода и редактирования данных. Обеспечена навигация, поиск, редактирование и фильтрация данных. В результате составлена спецификация и оформлен отчет.

СОДЕРЖАНИЕ

Введение.....5
1 Проектирование базы данных.....7
1.1 Выполнить проектирование БД. Использовать метод декомпозиции универсального отношения или метод ER-диаграмм. Определить структуру данных, типы данных и средства обеспечения целостности БД. 7
2 Построение БД.....10
2.1 Создание БД, создание таблиц БД, определение свойств полей. Создать таблицы, получившиеся в результате проектирования, описать свойства полей для каждой таблицы. Связать таблицы, обеспечив целостность данных.....10
2.2 Заполнение базы данных. Заполнить все таблицы данными, подобрав их таким образом, чтобы можно было проверить адекватность БД. Одну из таблиц оставить пустой и заполнить после создания для нее формы ввода. .11
3 Разработка интерфейса БД 14
3.1 Создание форм для ввода и редактирования данных. Для всех таблиц создать формы ввода и редактирования данных. 14
3.2 Работа с данными: навигация, поиск, редактирование, фильтрация. 17
3.3 Запросы. Создать запросы, количество и текст которых обусловлены условием задачи.....19
3.4 Создание системы оперативной справки. Используя любой ПП, создать краткую справку по работе с созданной программой. 19
4 Разработка технического задания.....27
Заключение.....30
Список используемых источников.....32
Приложение А: Листинг кода.....34

ВВЕДЕНИЕ

В современном мире информационные технологии проникли во все сферы деятельности человека. Образование не стало исключением: автоматизация процессов управления учебным заведением, учёта кадрового состава и контроля успеваемости позволяет существенно повысить эффективность работы образовательных организаций. Традиционные методы ведения документации — бумажные журналы, списки учителей и предметов — обладают рядом недостатков: они трудоёмки в обработке, подвержены риску утраты данных и не обеспечивают оперативного доступа к информации. Переход к цифровым решениям становится не просто желательным, а необходимым условием для современной образовательной среды.

Актуальность темы данной работы обусловлена потребностью в создании надёжного, структурированного и удобного инструмента для хранения и обработки данных об учителях, предметах, учениках и их успеваемости. Разработанный программный модуль позволяет автоматизировать ключевые процессы, сократить временные затраты на поиск и обработку информации, а также обеспечить её целостность и согласованность.

Целью учебной практики являлась разработка программного модуля «Школа: учителя, предметы, ученики, журнал успеваемости», реализующего функции хранения, обработки и представления данных об образовательном процессе.

Для достижения поставленной цели были решены следующие задачи:

– спроектирована структура базы данных с использованием метода ER-диаграмм;

– определены типы данных и свойства полей;

– создана база данных, таблицы и установлены связи между ними;

– выполнено заполнение базы данных тестовыми данными;

– разработан пользовательский интерфейс: созданы формы для ввода и редактирования данных;

– реализованы функции навигации, поиска, фильтрации и редактирования данных;

– составлена спецификация базы данных;

– оформлена пояснительная записка в соответствии с установленными требованиями.

Работа выполнялась в рамках прохождения учебной практики УП.11.01 по профессиональному модулю ПМ.11 «Разработка, администрирование и защита баз данных» на базе ГБ ПОУ РК «ФПТ» в группе ИСПП 23 2/9 в 2025 году.

Результатом практики стал готовый к эксплуатации программный модуль, отвечающий требованиям индивидуального задания и пригодный для использования в образовательном процессе.

Пояснительная записка оформлена согласно требованиям ГОСТ 7.32-2017 СИБИД «Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления (с Поправками)». Демонстрационный материал представлен в виде слайдов презентации.

1 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ

1.1 Выполнить проектирование БД. Использовать метод декомпозиции универсального отношения или метод ER-диаграмм. Определить структуру данных, типы данных и средства обеспечения целостности БД.

Для проектирования БД нам необходимо определить предметную область:

Описание предметной области: Автоматизация учета успеваемости в школе. Основные сущности: пользователи (администраторы, учителя, ученики), классы, предметы, связи класс-предмет-учитель, текущие оценки, итоговые оценки.

Пользователь имеет фамилию, имя, отчество, логин, пароль и роль. Класс имеет название и номер. Предмет имеет название. Связь класс-предмет-учитель определяет, какой учитель ведет какой предмет в каком классе. Текущая оценка содержит дату, оценку от 2 до 5 или Н, ссылку на ученика и ссылку на связь. Итоговая оценка содержит оценку от 2 до 5, ссылку на ученика и ссылку на связь.

Связи: один класс может содержать много учеников, один учитель может вести много предметов в разных классах, один ученик может иметь много текущих и итоговых оценок. Каждая оценка относится к конкретной связи класс-предмет-учитель. Ученик не может получить две оценки в один день по одному предмету. Итоговая оценка по предмету может быть только одна для каждого ученика.

Бизнес-правила: логин пользователя уникален, учитель может вести один предмет в одном классе только один раз, оценки только от 2 до 5 . Доступ к функциям системы определяется ролью: администратор управляет всем, учитель выставляет оценки по своим предметам, ученик только просматривает оценки.

В реляционную модель проектированной БД будут входить следующие сущности: Пользователи, классы, предметы, связи класс-предмет-учитель, текущие оценки, итоговые оценки.

Составим атрибуты для каждой сущности в таблицах 1.1-1.6.

Таблица 1.1 - Список атрибутов для сущности "Пользователи"

№ п.п	Ключевое поле	Имя	Наименование сущности	Тип данных	Ограничения
1	2	3	4	5	6
1	ПК	person_id	Идентификатор пользователя	SERIAL (integer)	PK
2	-	first_name	Имя	VARCHAR(50)	NOT NULL
3	-	last_name	Фамилия	VARCHAR(50)	NOT NULL
4	-	patronymic	Отчество	VARCHAR(50)	NOT NULL
5	-	login	Логин	VARCHAR(50)	UNIQUE
6	-	password	Пароль	VARCHAR(255)	NOT NULL
7	-	rights	Роль	VARCHAR(20)	NOT NULL

Таблица 1.2 - Список атрибутов для сущности "Классы"

№ п.п	Ключевое поле	Имя	Наименование сущности	Тип данных	Ограничения
1	2	3	4	5	6
1	ПК	class_id	Идентификатор класса	SERIAL (integer)	PK
2	-	class_name	Название класса	VARCHAR(10)	NOT NULL

Таблица 1.3 - Список атрибутов для сущности "Предметы"

№ п.п	Ключевое поле	Имя	Наименование сущности	Тип данных	Ограничения
1	2	3	4	5	6
1	ПК	subject_id	Идентификатор предмета	SERIAL (integer)	PK
2	-	title	Название предмета	VARCHAR(100)	NOT NULL

Таблица 1.4 - Список атрибутов для сущности "Связь класс-предмет-учитель"

№ п.п	Ключевое поле	Имя	Наименование сущности	Тип данных	Ограничения
1	2	3	4	5	6
1	ПК	cst_id	Идентификатор связи	SERIAL (integer)	ПК
2	ВК	class_id	Идентификатор класса	INTEGER	ВК
3	ВК	subject_id	Идентификатор предмета	INTEGER	ВК
4	ВК	teacher_id	Идентификатор учителя	INTEGER	ВК

Таблица 1.5 - Список атрибутов для сущности "Текущие оценки"

№ п.п	Ключевое поле	Имя	Наименование сущности	Тип данных	Ограничения
1	2	3	4	5	6
1	ПК	note_id	Идентификатор оценки	SERIAL (integer)	ПК
2	-	visit_date	Дата выставления	DATE	NOT NULL
3	-	note	Оценка	INTEGER	2-5, Н
4	ВК	cst_id	Идентификатор связи класс-предмет-учитель	INTEGER	ВК
5	ВК	fk_person_id	Идентификатор ученика	INTEGER	ВК

Таблица 1.6 - Список атрибутов для сущности "Итоговые оценки"

№ п.п	Ключевое поле	Имя	Наименование сущности	Тип данных	Ограничения
1	2	3	4	5	6
1	ПК	note_id	Идентификатор итоговой оценки	SERIAL (integer)	ПК
2	-	note	Оценка	INTEGER	2-5, Н/А
3	ВК	cst_id	Идентификатор связи класс-предмет-учитель	INTEGER	ВК
4	ВК	fk_person_id	Идентификатор ученика	INTEGER	ВК

Выполнили проектирование БД с помощью метода ER-диаграмм. База данных будет иметь 6 таблиц и связи многие ко многим.

2 ПОСТРОЕНИЕ БД

2.1 Создание БД, создание таблиц БД, определение свойств полей. Создать таблицы, получившиеся в результате проектирования, описать свойства полей для каждой таблицы. Связать таблицы, обеспечив целостность данных.

Перед созданием базы данных составим ER-диаграмму со свойствами полей и указанием ключевых полей. ER-диаграмма представлена на рисунке 2.1.

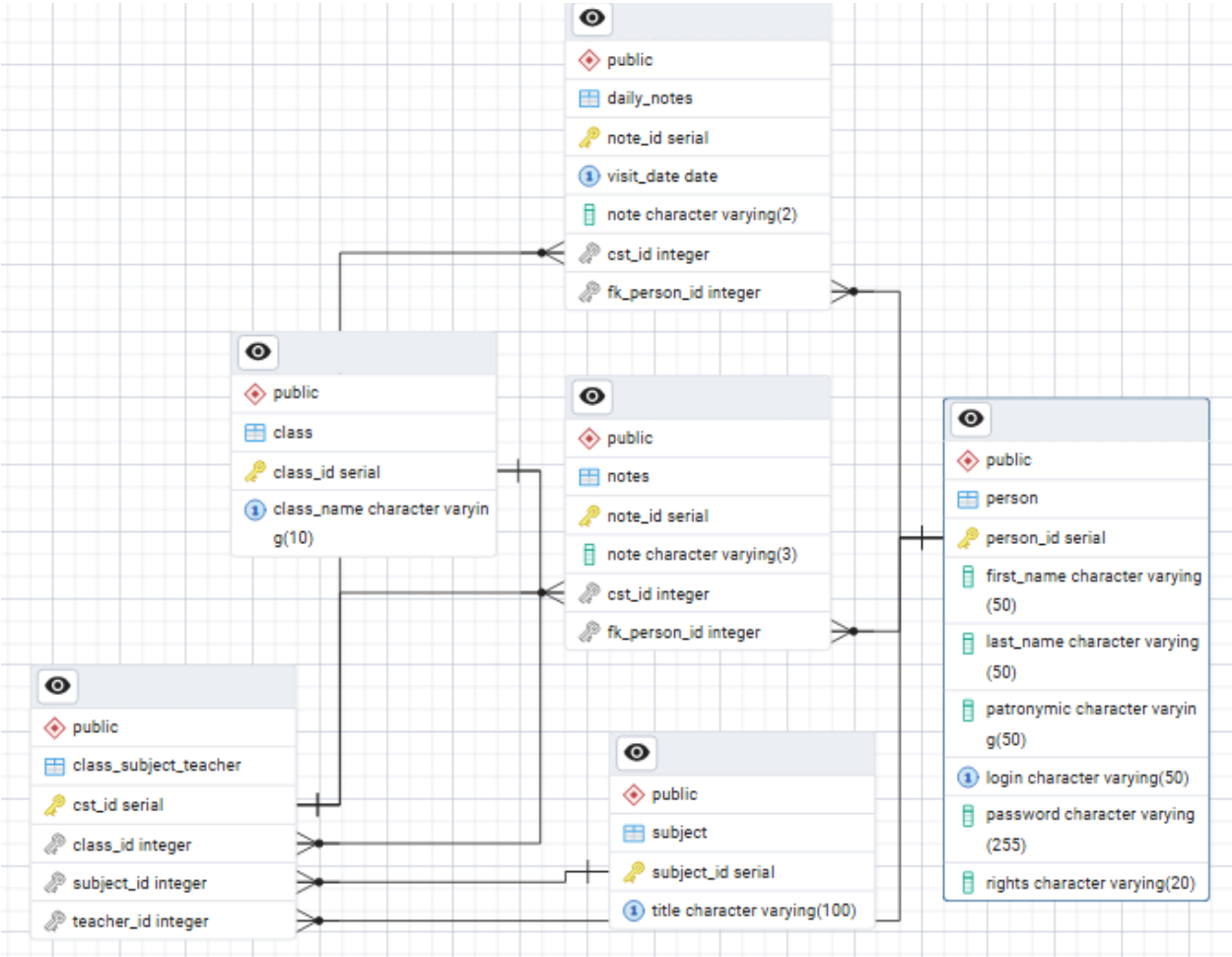

Рисунок 2.1 – ER-диаграмма

Для создания БД выбрали PostgreSql. По составленным ранее атрибутам создадим базу данных и таблицы. Созданная БД изображена на рисунке 2.2.

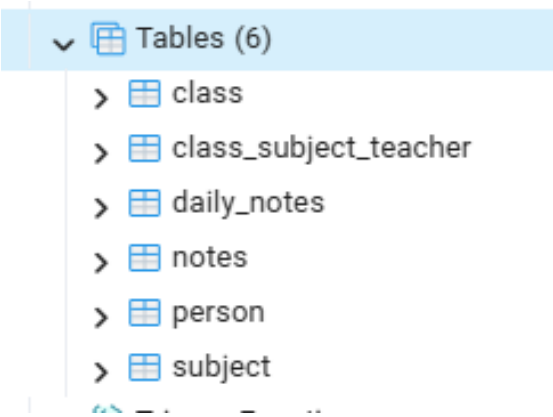

Рисунок 2.2 – Созданная база данных

2.2 Заполнение базы данных. Заполнить все таблицы данными, подобрав их таким образом, чтобы можно было проверить адекватность БД. Одну из таблиц оставить пустой и заполнить после создания для нее формы ввода.

Для заполнения базы данных определим данные таким образом, чтобы можно было проверить адекватность БД. Для этого необходимо определить несколько учителей, учеников и одного администратора. Для предметов берем типичные школьные предметы: математика, русский язык, английский язык, физика, литература, информатика, история и химия. Для таблицы классов выбрали три класса: 1А, 2Б, 3В. Составили связи и затем выставили оценки по предметам для проверки.

Затем с помощью команд INSERT заполним БД составленными данными. Таблицы с данными представлены на рисунках 2.3-2.7.

	person_id [PK] integer	first_name character varying (50)	last_name character varying (50)	patronymic character varying (50)	login character varying (50)	password character varying (255)	rights character varying (20)
1	1	Иван	Иванов	Иванович	admin	admin123	Администратор
2	2	Анна	Петрова	Сергеевна	teacher_math	pass123	Учитель
3	3	Елена	Смирнова	Алексеевна	teacher_russian	pass123	Учитель
4	4	Ольга	Кузнецова	Владимировна	teacher_english	pass123	Учитель
5	5	Дмитрий	Соколов	Николаевич	teacher_physics	pass123	Учитель
6	6	Алексей	Федоров	Дмитриевич	student1	pass123	Ученик
7	7	Мария	Ильина	Андреевна	student2	pass123	Ученик
8	8	Кирилл	Васильев	Павлович	student3	pass123	Ученик
9	9	Дарья	Морозова	Игоревна	student4	pass123	Ученик
10	10	Егор	Новиков	Александрович	student5	pass123	Ученик
11	11	София	Лебедева	Максимовна	student6	pass123	Ученик

Рисунок 2.3 – Заполненная таблица person

	class_id [PK] integer	class_name character varying (10)	year integer
1	1	1А	2024
2	2	2Б	2024
3	3	3В	2024

Рисунок 2.4 – Заполненная таблица class

	subject_id [PK] integer	title character varying (100)
1	1	Математика
2	2	Русский язык
3	3	Английский язык
4	4	Физика
5	5	Литература
6	6	Информатика
7	7	История
8	8	Химия

Рисунок 2.5 – Заполненная таблица subject

	cst_id [PK] integer	class_id integer	subject_id integer	teacher_id integer
1	1	1	1	2
2	2	1	2	3
3	3	1	3	4
4	4	1	5	3
5	5	2	1	2
6	6	2	2	3
7	7	2	3	4
8	8	2	4	5
9	9	2	6	5
10	10	3	1	2
11	11	3	2	3
12	12	3	3	4
13	13	3	7	3
14	14	3	8	5

Рисунок 2.6 – Заполненная таблица class_subject_teacher

	note_id [PK] integer	datetime date	note integer	cst_id integer	fk_person_id integer
1	1	2024-03-...	4	1	6
2	2	2024-03-...	5	2	6
3	3	2024-03-...	3	3	6
4	4	2024-03-...	5	4	6
5	5	2024-03-...	4	1	7
6	6	2024-03-...	4	2	7
7	7	2024-03-...	5	3	7
8	8	2024-03-...	5	4	7
9	9	2024-03-...	3	5	8
10	10	2024-03-...	4	6	8
11	11	2024-03-...	4	7	8
12	12	2024-03-...	5	8	8
13	13	2024-03-...	4	5	9
14	14	2024-03-...	5	6	9
15	15	2024-03-...	4	7	9
16	16	2024-03-...	5	9	9
17	17	2024-03-...	4	10	10
18	18	2024-03-...	3	11	10

Рисунок 2.7 – Заполненная таблица daily_notes

Таблицу notes оставим пустой для заполнения ее после создания формы ввода.

3 РАЗРАБОТКА ИНТЕРФЕЙСА БД

3.1 Создание форм для ввода и редактирования данных. Для всех таблиц создать формы ввода и редактирования данных.

Для создания интерфейса базы данных используем C# и Visual Studio. Создадим форму авторизации, она изображена на рисунке 3.1.

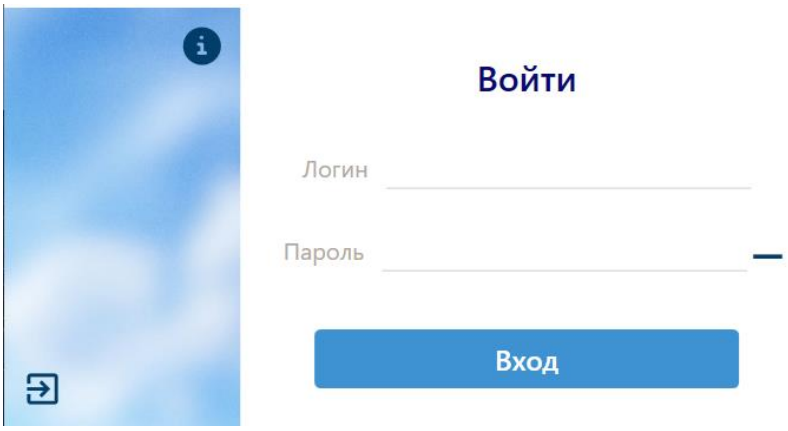

Рисунок 3.1 – Форма авторизации

Для каждой таблицы создадим формы ввода и редактирования данных. Для таблицы person создали форму “Создание нового профиля” (рис.3.2)

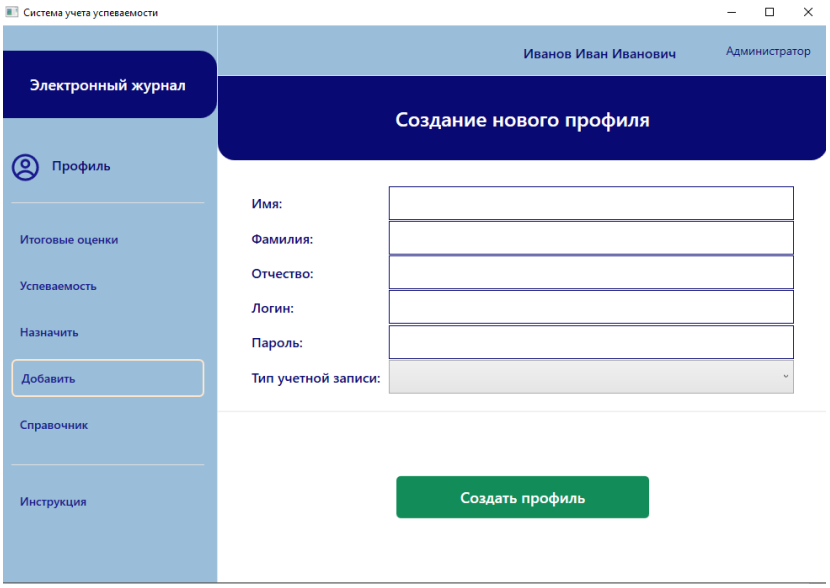

Рисунок 3.2 -Создание нового профиля

Для таблицы notes создали форму “Итоговые оценки” (рис.3.3).

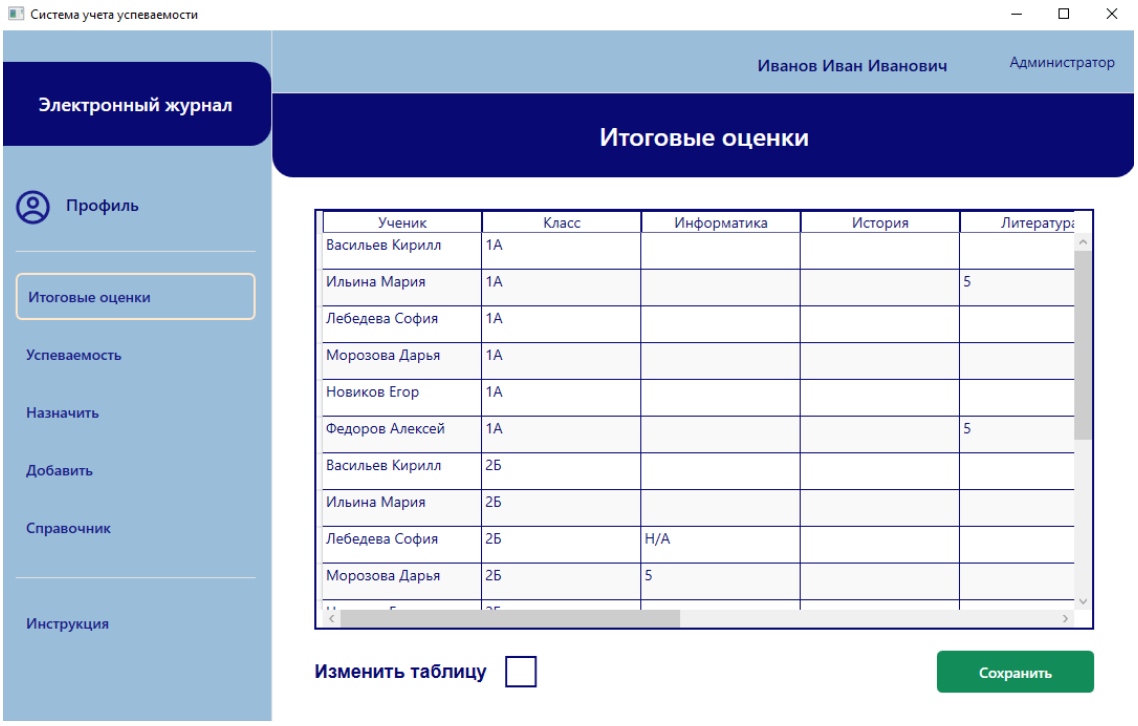

Рисунок 3.3 – Итоговые оценки

Для таблицы daily_notes создали форму “Успеваемость” (рис.3.4).

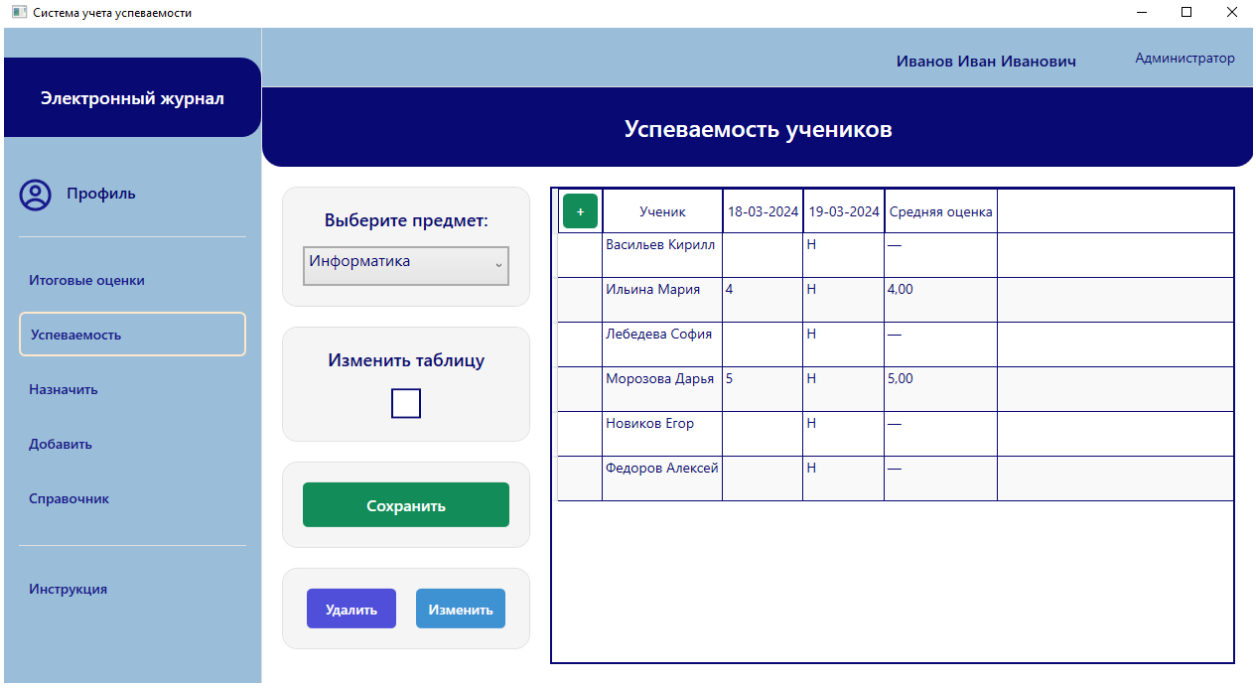

Рисунок 3.4 – Успеваемость

Для таблиц предметы и классы создали форму “Справочник”, где с помощью ComboBox можно выбрать интересующую таблицу (рис.3.5-3.6).

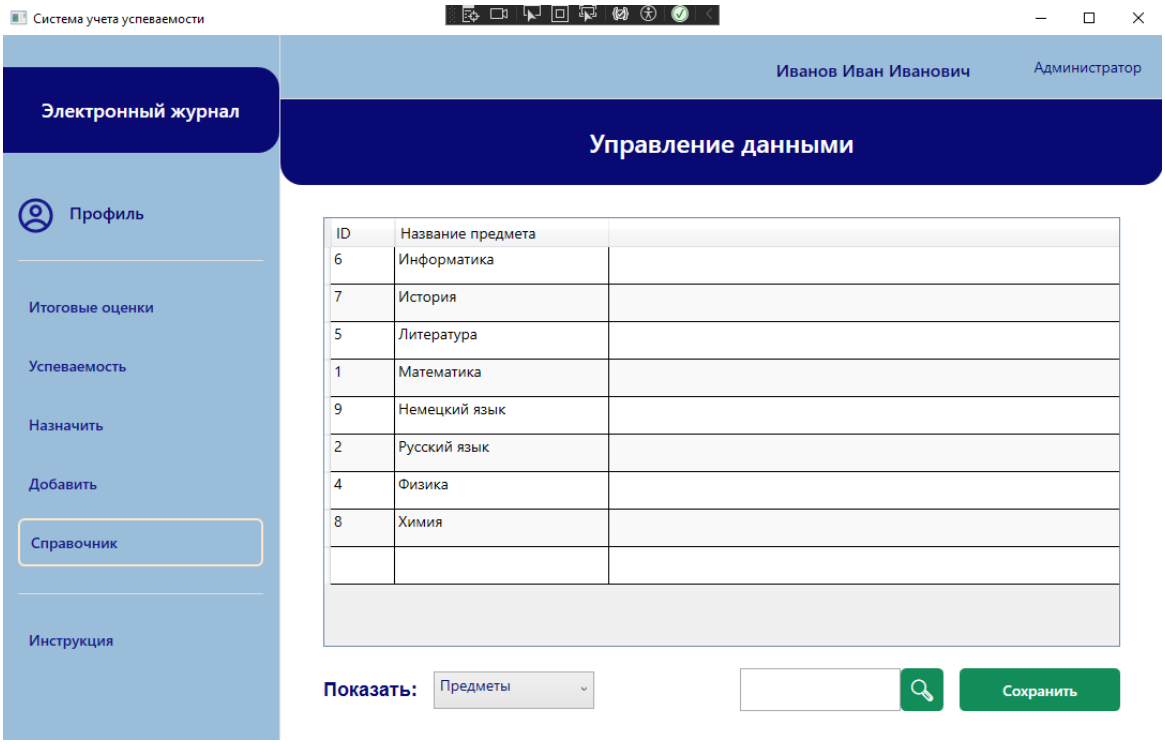

Рисунок 3.5 – Справочник для предметов

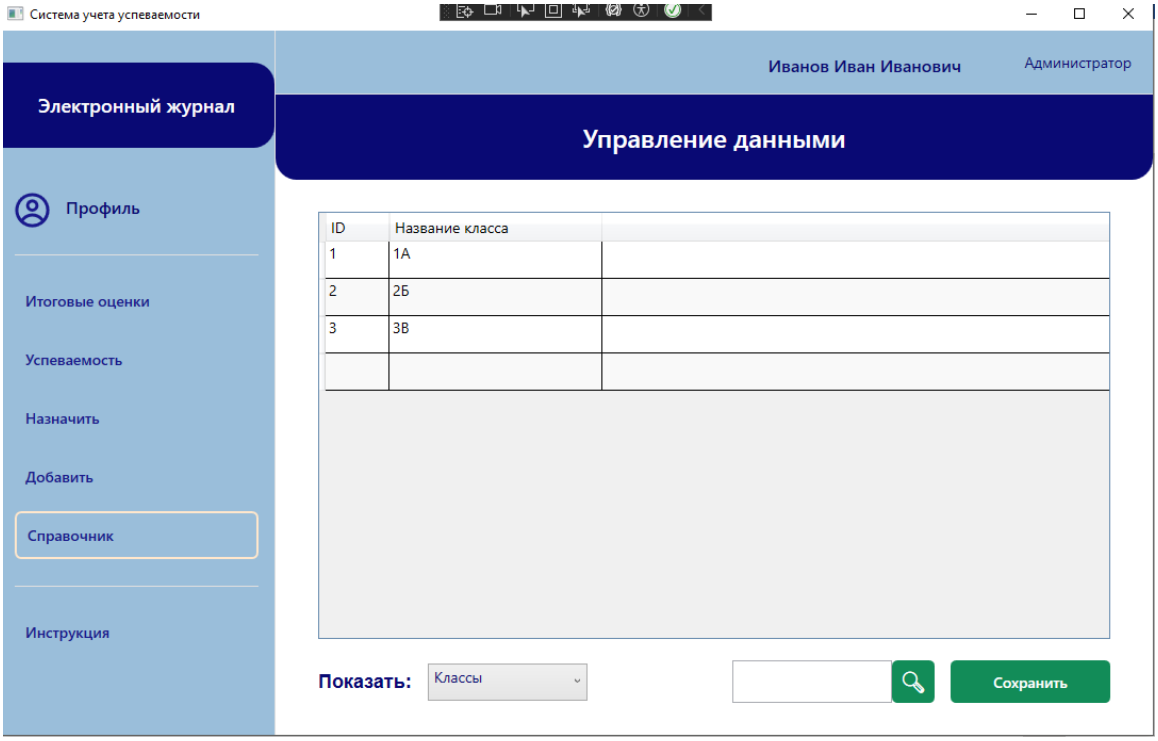

Рисунок 3.6 - Справочник для классов

3.2 Работа с данными: навигация, поиск, редактирование, фильтрация.

Далее мы реализовали навигацию с помощью кнопок на основной форме (рис.3.7).

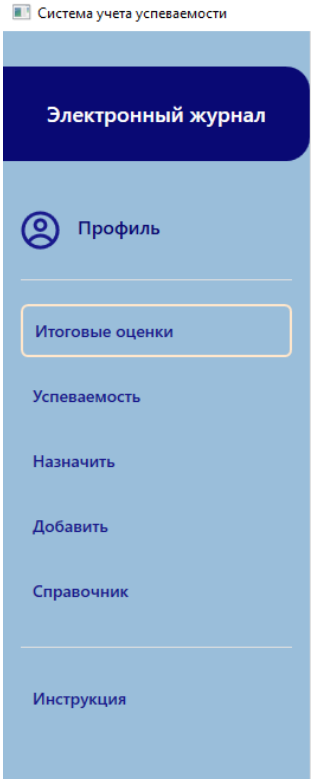

Рисунок 3.7 – Навигация

Далее реализуем поиск. Для этого на формы с данными поместим TextBox и кнопку, отвечающие за поиск (рис.3.8).

Рисунок 3.8 – Кнопка “Поиск”

В процессе разработки использовался класс database для выполнения запросов к PostgreSQL через методы DataQuery (получение данных) и ExecuteNonQuery (INSERT, UPDATE, DELETE). Данные в памяти хранятся в DataTable, который отслеживает изменения строк, и DataView для фильтрации

через RowFilter. Визуальное отображение обеспечивает DataGrid с поддержкой редактирования и добавления записей. Для безопасности применяются параметризованные запросы с NpgsqlParameter, защищающие от SQL-инъекций. Реализация поиска представлена на рисунке 3.9.

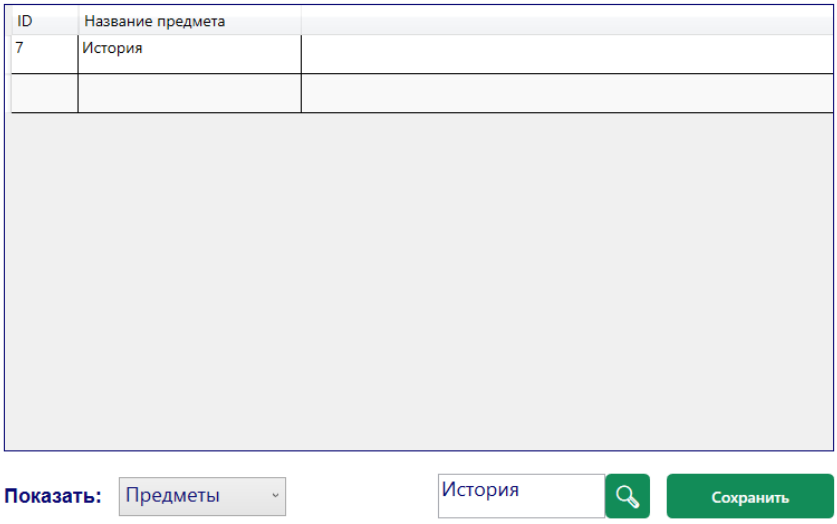

Рисунок 3.9 – Поиск

Редактирование и сортировка данных реализована с помощью SQL-запросов к базе данных и представлены на рисунке 3.10.

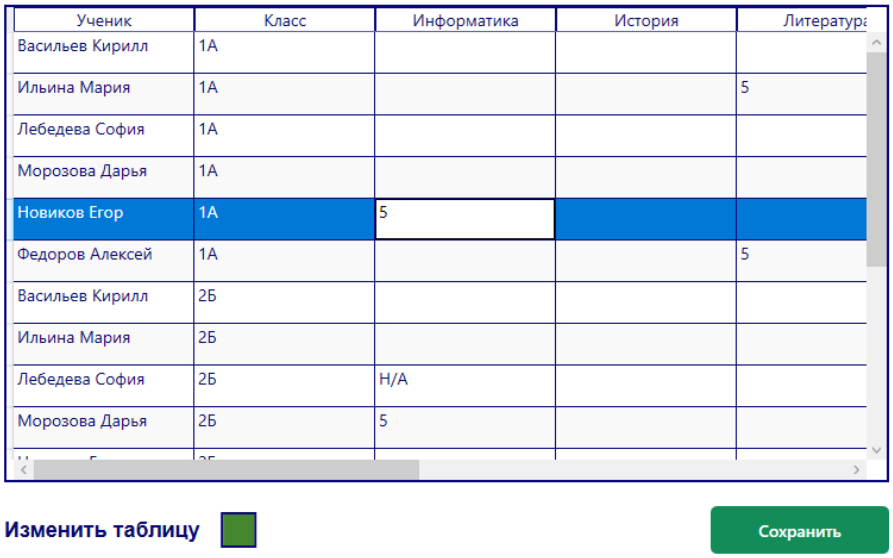

Рисунок 3.10 – Редактирование и сортировка данных

3.3 Запросы. Создать запросы, количество и текст которых обусловлены условием задачи.

Для выполнения индивидиуального задания нам необходимо составить запросы к базе данных с помощью языка SQL для выбора информации и заполнения таблиц. SQL-запросы представлены на рисунках 3.11-3.12.

```
string query = @"SELECT person_id AS ""ID"",
                        first_name AS ""Имя"",
                        last_name AS ""Фамилия"",
                        patronymic AS ""Отчество"",
                        login AS ""Логин"",
                        rights AS ""Права""
                  FROM person
                  WHERE rights = 'Ученик'
                  ORDER BY last_name, first_name;";
```

Рисунок 3.11 – SQL-запрос SELECT

```
string title = row["Название предмета"].ToString();
string query = "INSERT INTO subject (title) VALUES (@title);";
```

Рисунок 3.12 – SQL-запрос INSERT

3.4 Создание системы оперативной справки. Используя любой ПП, создать краткую справку по работе с созданной программой.

При запуске программы двойным кликом левой кнопкой мыши запускается окно авторизации, в которое необходимо ввести логин и пароль, полученные от администратора (рис.3.13).

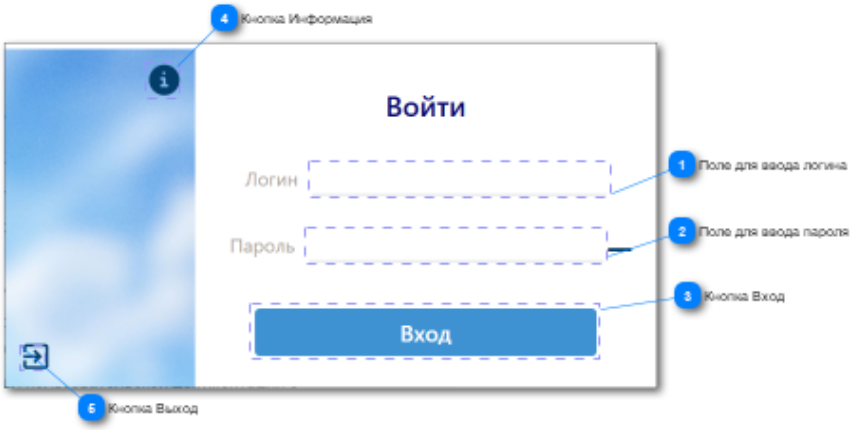

Рисунок 3.13 – Окно авторизации

Также в окне авторизации есть кнопка “Выход” и кнопка, выдающая информацию о том, как зайти в акканут.

В главном меню у администратора есть 7 пунктов на выбор: профиль, итоговые оценки, успеваемость, назначить, добавить, справочник и инструкция. Сверху окна отображается роль и имя пользователя (рис.3.14).

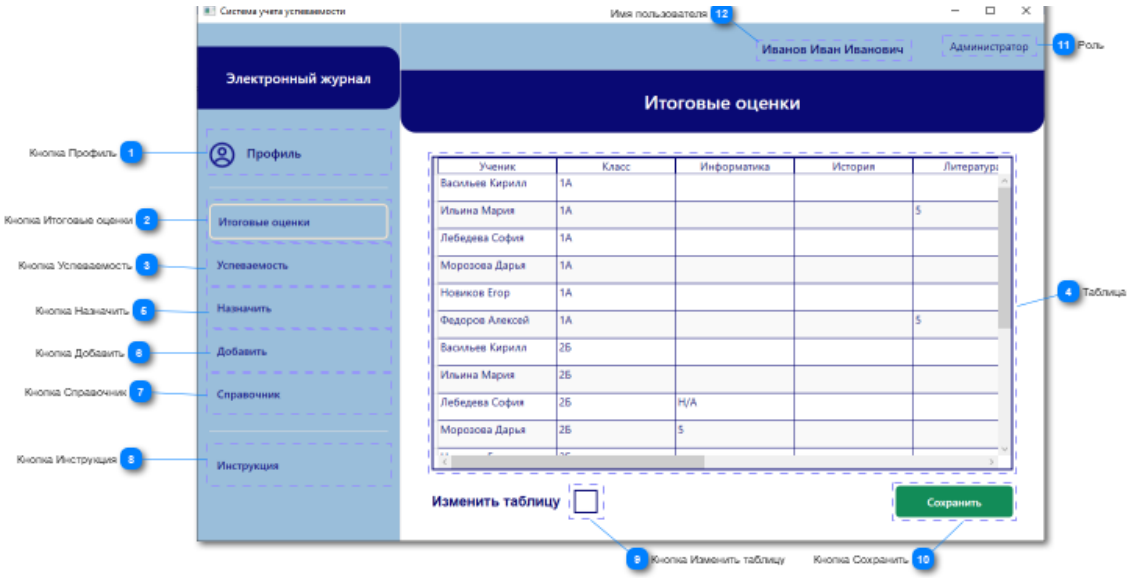

Рисунок 3.14 – Главное окно со страницей “Итоговые оценки”

Кнопка “Профиль” перемещает на страницу “Профиль”. На странице “Профиль” находится информация о пользователе, а на кнопку “Изменить данные” можно сменить логин и пароль (рис.3.15).

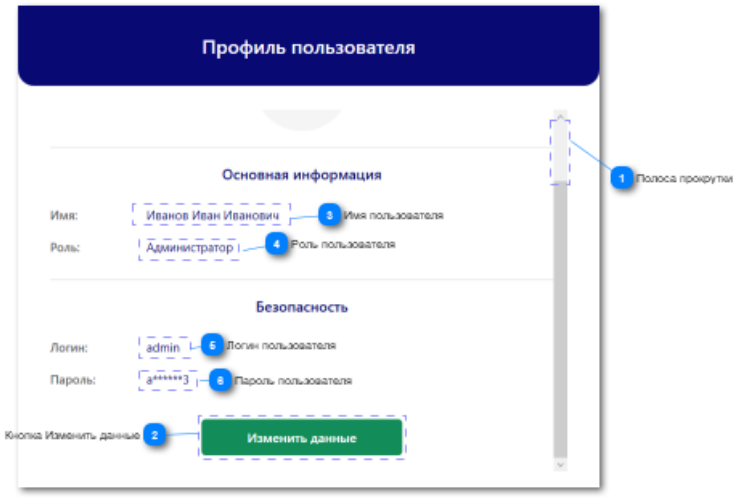

Рисунок 3.15 – Страница “Профиль”

Кнопка “Итоговые оценки” перемещает на страницу “ Итоговые оценки”. На этой странице находится таблица с итоговыми оценками всех учеников, а также кнопка “Сохранить” и кнопка “Изменить таблицу”. Кнопка “Сохранить” сохраняет данные, а кнопка “Изменить таблицу” является переключателей для изменения или просмотра таблицы (рис.3.14).

Кнопка “Успеваемость” перемещает на страницу “Успеваемость учеников”, где находится выпадающий список со школьными предметами. После выбора предмета в таблице отображаются оценки и посещаемость учеников по выбранному предмету (рис.3.16-3.19).

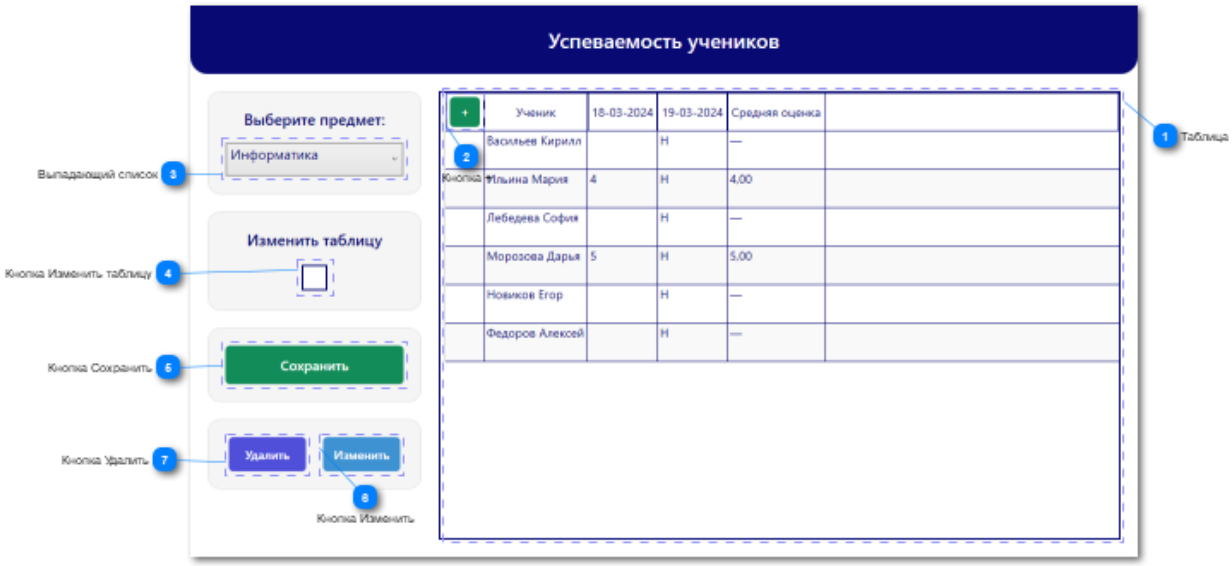

Рисунок 3.16 - Страница “Успеваемость учеников”

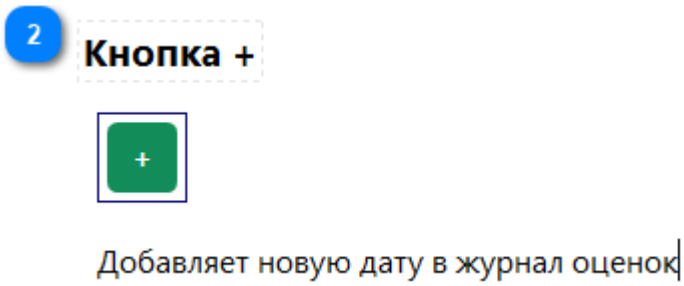

Рисунок 3.17 – Кнопка “+”

Кнопка “Изменить таблицу” позволяет переключить просмотр и изменение таблицы. Кнопка “Сохранить” сохраняет изменения в базу данных.

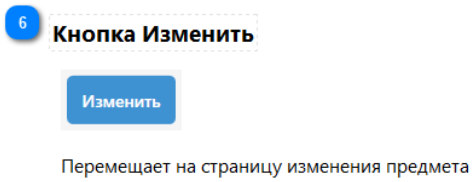

Рисунок 3.18 – Кнопка “Изменить”

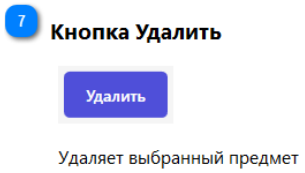

Рисунок 3.19 – Кнопка “Удалить”

Кнопка “Назначить” перемещает на форму “Назначение учителей на предметы”. На странице представлены 3 выпадающих списка с учителями, классами и предметами, а также 3 кнопки: назначить, новая дисциплина, новый профиль (рис.3.20).

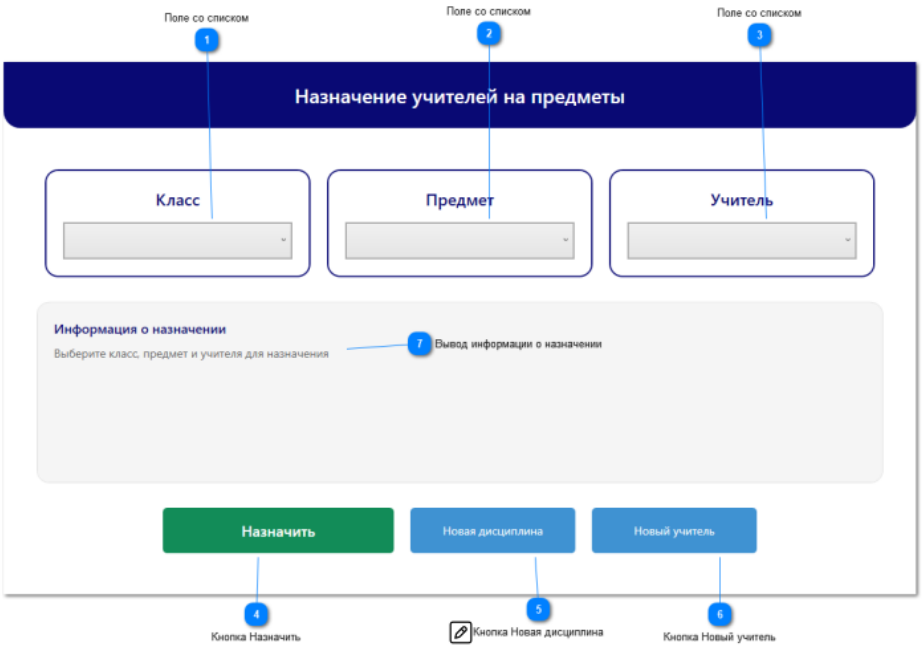

Рисунок 3.20 – Страница “Назначение учителей на предметы”

Кнопка “Назначить” назначает учителя вести предмет у выбранного класса. Кнопка “Новая дисциплина” перемещает на страницу создания дисциплин, а кнопка “Новый учитель” перемещает на страницу создания профилей.

Кнопка “Добавить” перемещает на форму “Добавление данных”, где указана инструкция к добавлению данных и две кнопки “Новая дисциплина” и “Новый профиль” (рис.3.21).

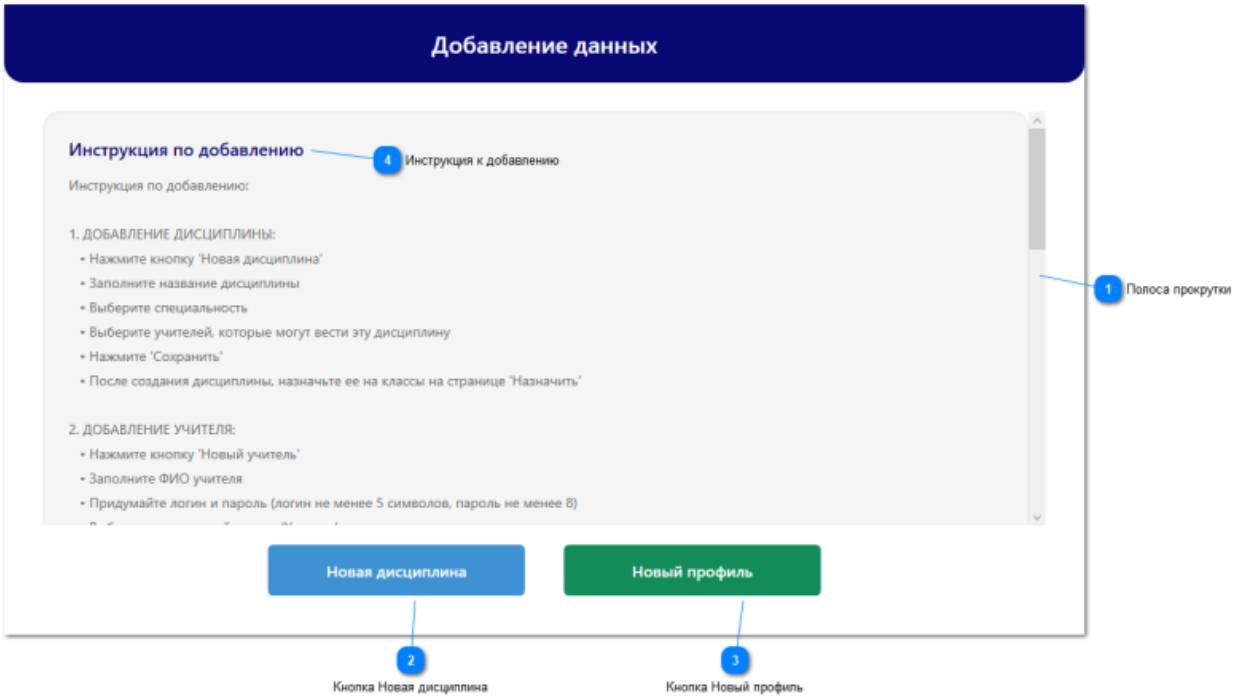

Рисунок 3.21 – Страница “Добавление данных”

Кнопка “Новая дисциплина” перемещает на страницу создания новой дисциплины. Кнопка “Добавление данных” перемещает на страницу создания нового профиля.

Кнопка “Справочник” перемещает на страницу с данными всех таблиц базы данных (рис.3.22).

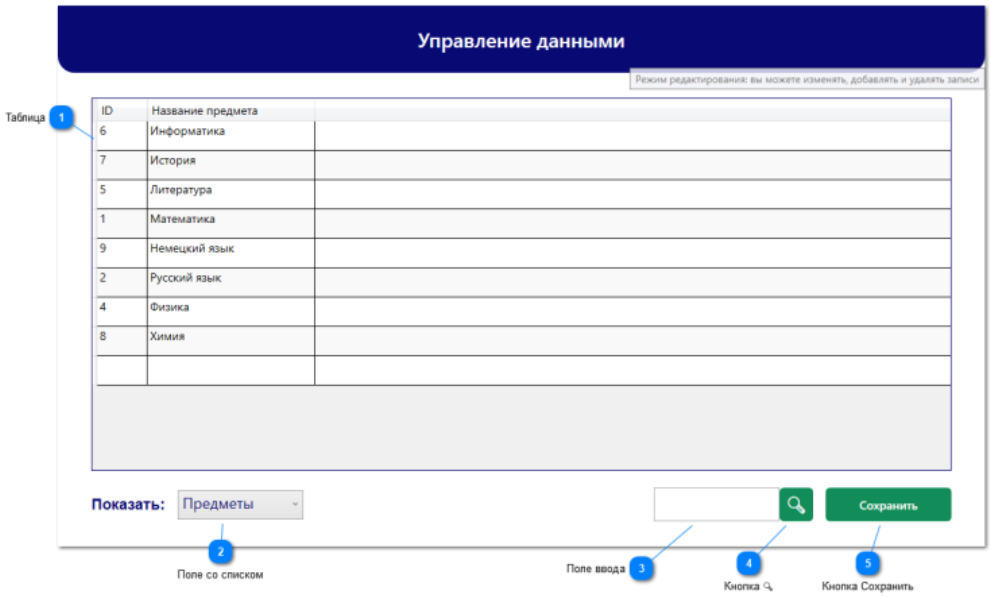

Рисунок 3.22 – Страница “Справочник”

В выпадающем списке можно выбрать таблицу для просмотра или изменения. В редактируемое поле вводится текст для поиска в таблице и после нажатия на кнопку “Поиск” производится поиск по выбранной таблице. Кнопка “Сохранить” сохраняет изменения в базу данных.

Кнопка “Инструкция” сообщает инструкцию по использованию программы в отдельном окне (рис.3.23).

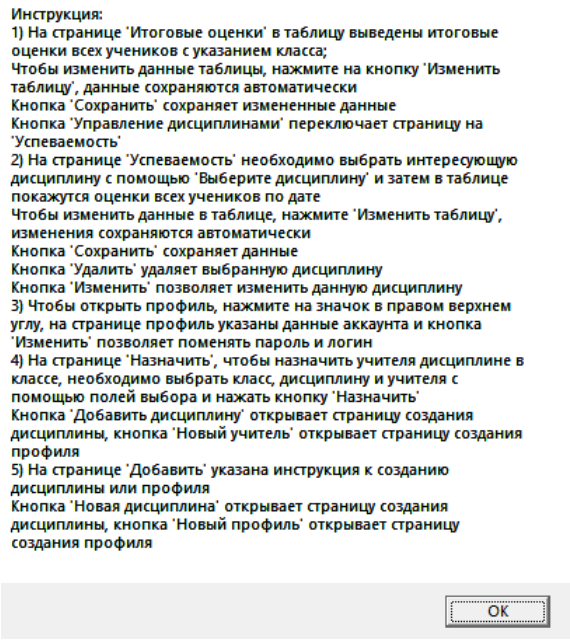

Рисунок 3.23 – Инструкция

На страницах “Новая дисциплина” и “Новый преподаватель” изображены поля для ввода данных, и кнопка “Создать” (рис.3.24-3.25).

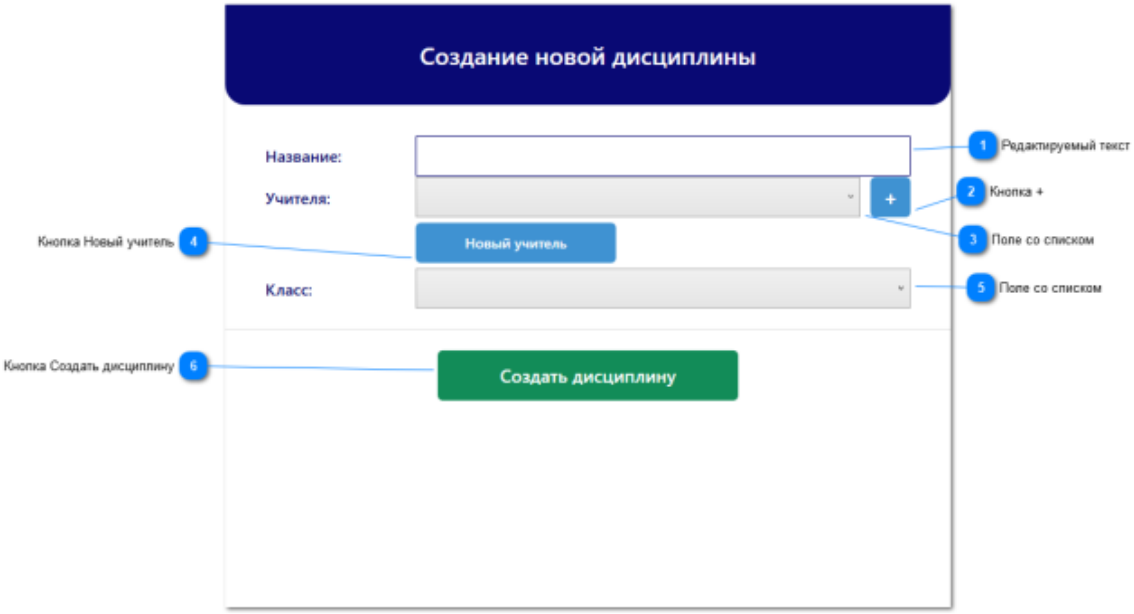

Рисунок 3.24 – Страница “Создание новой дисциплины”

Кнопка “Новый учитель” перемещает на страницу “Создание нового профиля”, кнопка “+” добавляет несколько учителей вести дисциплину, в выпадающих списках выбираются уже зарегистрированные учителя и классы.

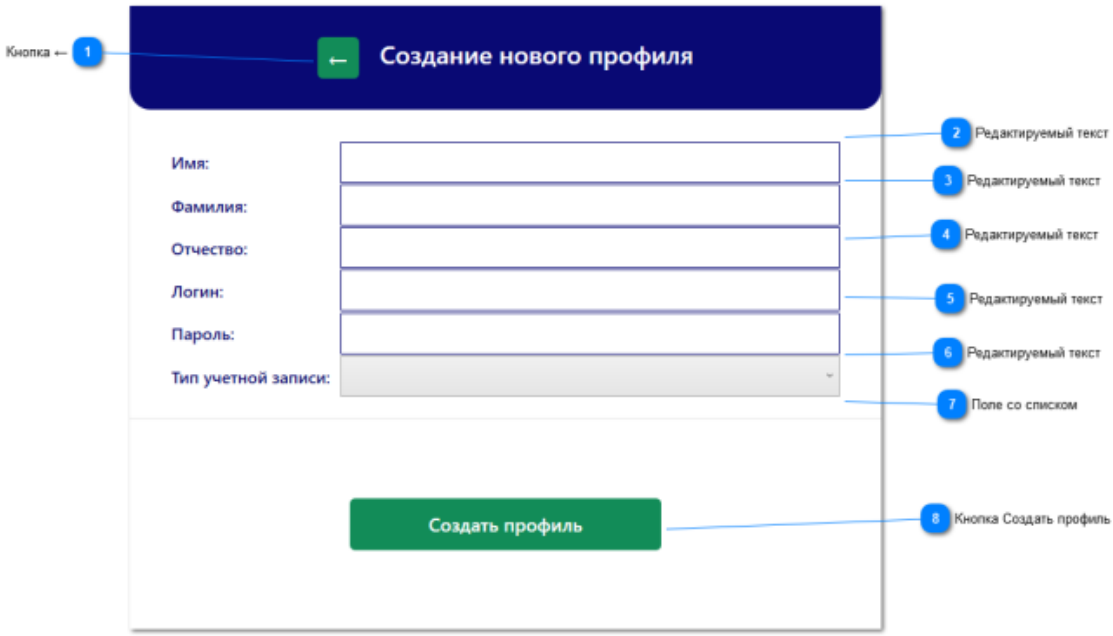

Рисунок 3.25 – Страница “Создание нового профиля”

Кнопка “Создать профиль” создает новый профиль. В соответствующие поля ввода необходимо ввести информацию о профиле. В выпадающем списке необходимо выбрать тип учетной записи: ученик, учитель, администратор. Если выбирается учитель, то на выбор появляется выпадающий список существующих предметов и кнопка “Добавить предмет” и “Новый предмет”, которые позволяют добавить несколько предметов учителю или создать новую дисциплину соответственно. Кнопка “Вернуться” позволяет вернуться на форму “Создание новой дисциплины”.

Учителю доступен просмотр и изменение отметок только по своим дисциплинам, а также добавление дат.

Ученику доступен только просмотр своих оценок.

Просмотр справочника доступен всем ролям.

4 РАЗРАБОТКА ТЕХНИЧЕСКОГО ЗАДАНИЯ

4.1. Общие сведения

4.1.1. Наименование программного продукта: «Школа: учителя, предметы, ученики, журнал успеваемости».

4.1.2. Назначение программного продукта: автоматизация учета образовательного процесса, включая ведение базы данных участников образовательного процесса, контроль успеваемости, назначение преподавателей на дисциплины.

4.1.3. Заказчик: ГБ ПОУ РК «ФПТ».

4.1.4. Разработчик: студент группы ИСПП 23 2/9.

4.2. Назначение разработки

4.2.1. Программный модуль предназначен для автоматизации процессов учета успеваемости в образовательном учреждении.

4.2.2. Основными целями разработки являются:

– централизованное хранение информации об учениках, учителях, классах, предметах и успеваемости;

– обеспечение оперативного доступа к данным образовательного процесса;

– разграничение прав доступа в зависимости от роли пользователя;

– сокращение временных затрат на обработку и поиск информации;

– обеспечение целостности и согласованности данных.

4.3. Требования к программному продукту

4.3.1. Требования к функциональным характеристикам

4.3.1.1. Система должна обеспечивать аутентификацию пользователей по логину и паролю.

4.3.1.2. Система должна обеспечивать визуальную оболочку для базы данных предметной области:

4.3.1.3. Система должна обеспечивать разграничение прав доступа в соответствии с ролями: администратор, учитель, ученик.

4.3.1.4. Администратор должен иметь возможность:

– создавать, редактировать и удалять учетные записи пользователей;

– создавать, редактировать и удалять предметы;

– создавать, редактировать и удалять классы;

– назначать преподавателей на предметы и классы;

– просматривать и редактировать успеваемость всех учеников.

4.3.1.5. Учитель должен иметь возможность:

– просматривать список назначенных ему предметов и классов;

– выставлять текущие и итоговые оценки ученикам по своим предметам;

– просматривать успеваемость учеников.

4.3.1.6. Ученик должен иметь возможность:

– просматривать свои текущие и итоговые оценки;

4.3.1.7. Система должна обеспечивать ввод, редактирование и удаление данных с помощью графического интерфейса.

4.3.1.8. Система должна обеспечивать поиск данных по заданным критериям.

4.3.1.9. Система должна обеспечивать фильтрацию данных с использованием выпадающих списков.

4.3.1.10. Система должна обеспечивать сортировку данных в таблицах.

4.3.1.11. Система должна автоматически сохранять изменения в базе данных.

4.3.2. Требования к надежности

4.3.2.1. Система должна корректно обрабатывать ошибки подключения к базе данных с выводом соответствующих сообщений.

4.3.2.2. Система должна обеспечивать целостность данных на уровне базы данных с использованием первичных и внешних ключей.

4.3.2.3. Система должна предотвращать удаление записей, имеющих связанные данные, с предварительным предупреждением пользователя.

4.3.3. Требования к информационной безопасности

4.3.3.1. Доступ к системе должен осуществляться только после успешной аутентификации.

4.3.3.2. Интерфейс системы должен скрывать элементы управления, недоступные пользователю в соответствии с его ролью.

4.3.4. Требования к техническому обеспечению

4.3.4.1. Операционная система: Windows 10 или выше.

4.3.4.2. Процессор: Intel Core i3 или аналогичный.

4.3.4.3. Оперативная память: не менее 4 ГБ.

4.3.4.4. Свободное место на жестком диске: не менее 500 МБ.

4.3.4.5. Сервер баз данных: PostgreSQL 14 или выше.

4.3.5. Требования к программному обеспечению

4.3.5.1. Среда разработки: Microsoft Visual Studio 2022.

4.3.5.2. Платформа: .NET Framework 4.7.2 или .NET Core.

4.3.5.3. Язык программирования: C#.

4.3.5.4. Технология пользовательского интерфейса: WPF.

4.3.5.5. Библиотека для работы с PostgreSQL: Npgsql.

ЗАКЛЮЧЕНИЕ

В ходе выполнения индивидуального задания в рамках учебной практики УП.11.01 по профессиональному модулю ПМ.11 «Разработка, администрирование и защита баз данных» был разработан программный модуль «Школа: учителя, предметы, ученики, журнал успеваемости». Разработка выполнена в соответствии с утвержденным техническим заданием и требованиями образовательного учреждения.

На первом этапе работы проведен анализ предметной области, в ходе которого выявлены основные сущности образовательного процесса: пользователи, классы, предметы, связи класс-предмет-учитель, текущие оценки и итоговые оценки. Определены бизнес-правила, обеспечивающие корректное функционирование системы, включая уникальность логинов пользователей, ограничения на выставляемые оценки и невозможность получения учеником двух оценок по одному предмету в один день.

На этапе проектирования базы данных использован метод ER-диаграмм, позволивший определить структуру данных, установить связи между таблицами и обеспечить целостность данных. Созданы таблицы person, class, subject, class_subject_teacher, daily_notes и notes с использованием системы управления базами данных PostgreSQL. Для каждой таблицы определены первичные и внешние ключи, установлены ограничения целостности. База данных заполнена тестовыми данными для проверки функциональности.

Разработка программного интерфейса выполнена с использованием языка C# и технологии WPF в среде Microsoft Visual Studio 2022. Созданы следующие формы: авторизации, итоговые оценки, успеваемость, назначение учителей, создание новой дисциплины, создание нового профиля, справочник. Реализованы функции навигации между формами, поиска и фильтрации данных, редактирования и сортировки информации в таблицах.

Реализована система разграничения прав доступа. Администратор имеет полный доступ ко всем функциям системы, включая создание, редактирование и удаление любых записей. Учитель может выставлять оценки только по назначенным ему предметам и классам. Ученик имеет возможность только просматривать свои оценки.

Для обеспечения безопасности данных использованы параметризованные запросы с применением NpgsqlParameter, что исключает возможность SQL-инъекций. Взаимодействие с базой данных осуществляется через специально разработанный класс database, инкапсулирующий методы DataQuery для получения данных и ExecuteNonQuery для выполнения операций INSERT, UPDATE, DELETE.

В процессе разработки проведено функциональное тестирование, включающее проверку работы всех форм, корректность выполнения операций с данными, правильность разграничения прав доступа, обработку ошибочных ситуаций. Результаты тестирования подтвердили соответствие программного модуля установленным требованиям.

Разработанное программное обеспечение позволяет эффективно решать задачи автоматизации образовательного учета: обеспечивает централизованное хранение информации, оперативный доступ к данным, удобный интерфейс для работы с оценками и назначениями преподавателей, формирование отчетности. Система обладает интуитивно понятным интерфейсом, снижающим затраты на обучение персонала.

Разработанный программный модуль может быть использован в образовательных учреждениях для автоматизации процессов контроля успеваемости и учета данных об участниках образовательного процесса, что позволит сократить временные затраты на ведение документации и повысить прозрачность учебного процесса. Система обладает потенциалом для дальнейшего развития, включая расширение функциональности, интеграцию с другими информационными системами и добавление аналитических инструментов для прогнозирования успеваемости.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32-2017 СИБИД Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. - Введ. 01.07.2018.
2. Крёнке, Д. Теория и практика построения баз данных. 10-е изд./ Д. Крёнке. — СПб.: Питер, 2023. — 850 с.: ил.
3. Шумаков П.В. ADO.NET и создание приложений баз данных в среде Microsoft Visual Studio.NET/ П.В. Шумаков. –: М.: ДИАЛОГ-МИФИ, 2023. – 300 с.
4. Либерти Д. Программирование на C#/ Д. Либерти. – М.: Издательский дом «Символ-Плюс», 2023. – 720 с.
5. Рихтер Д. Программирование на платформе Microsoft .NET Framework/ Д. Рихтер. – 2-е изд., перераб. и доп. – СПб.: Русская редакция, 2023. – 550 с.
6. Дейт, К. Дж. SQL и реляционная теория/ К. Дж. Дейт. – 3-е изд. – М.: Издательский дом «Вильямс», 2024. – 500 с.
7. Visual Studio 2022 [Электронный ресурс]. – 2003-2025. - Электрон дан. — Режим доступа: https://www.visualstudio.com/ru-ru/visual-studio-homepage-vs.aspx (дата обращения: 23.09.2024)
8. Вахрин, П. Методика подготовки и процедура защиты дипломных работ по финансовым и экономическим специальностям: учеб. пособие /П. Вахрин. – 6-е изд., перераб. и доп. - М.: Дашков и Ко, 2024. - 160 с.
9. Вирт, Никлаус. Алгоритмы и структуры данных/ Н. Вирт. – 3-е изд. – СПб.: БХВ-Петербург, 2023. – 300 с.
10. Рихтер, Дж. Программирование на платформе Microsoft .NET Framework 8.0 на языке C#/ Дж. Рихтер. – 4-е издание. – СПб.: Питер, 2024. – 450 с.: ил.

11. Микелсен К. Язык программирования C#. Лекции и упражнения/ К. Микелсен. – 3-е изд. – М.: Издательский дом «ДиаСофт», 2024. – 700 с.

12. Петзольд Ч. Программирование для Microsoft Windows на C#. В 2 томах/ Ч. Петзольд. – 3-е изд. – СПб.: Русская редакция, 2023. – 600 с.

13. Троелсен Э. C# и платформа .NET. Библиотека программиста/ Э. Троелсен – 8-е изд. – СПб.: Питер, 2024. – 850 с.

14. Тестирование программного обеспечения [Электронный ресурс] / Информация о тестировании программного обеспечения. – 2003-2025. - Электрон дан. – Режим доступа: https://www.softwaretestinghelp.com/ (дата обращения: 15.03.2025)

15. Фримен Э., Робсон Э., Бейтс Б., Сьерра К. Паттерны проектирования/ Э. Фримен, Э. Робсон, Б. Бейтс, К. Сьерра. – 3-е издание. – СПб.: Питер, 2023. – 680 с.

16. Бен-Ари М. Понимание программирования на C#/ М. Бен-Ари. – 4-е изд. – СПб.: Питер, 2024. – 550 с.

17. Албахари Д., Албахари Б. C# 12.0 и .NET 8.0. Карманный справочник/ Д. Албахари, Б. Албахари. – М.: Вильямс, 2023. – 1100 с.

18. Тейер, Т. Надежность программного обеспечения / Т. Тейер, М. Липов, Э. Нельсон. – 2-е изд. – М.: Мир, 2024. – 350 с.

19. Тестирование модуля [Электронный ресурс] / Информация о тестировании модуля. – 2003-2025. - Электрон дан. – Режим доступа: https://www.guru99.com/unit-testing-guide.html (дата обращения: 15.03.2025)

20. Бондарь А. Microsoft SQL Server 2025 // А. Бондарь. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2025. – 560 с.

Приложение А:

Листинг кода

```
using Npgsql;
using Syncfusion.Windows.Shared;
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Windows;
namespace WpfApp2
{
/// <summary>
/// Логика взаимодействия для MainWindow.xaml
/// </summary>
public partial class Autorization : Window
{
public Autorization()
{
InitializeComponent();
}
private string login;
private string password;
database db = new database();
bool mask = true;
private void Button_Click_1(object sender, RoutedEventArgs e)
{
try
{
Error.Visibility = Visibility.Visible;
login = loginText.Text;
password = parolText.Text;
if ((check_SpecialChars(login) || check_Space(login)) ||
check_Space(password)) return;
else Check_data_exists(login, password);
}
catch (Exception ex)
{
MessageBox.Show(ex.Message);
}
}
private void Check_data_exists(string login, string password)
{
try
{
IsFalse.Visibility = Visibility.Collapsed;
IsParol.Visibility = Visibility.Collapsed;
Err.Visibility = Visibility.Collapsed;
IsAll.Visibility = Visibility.Collapsed;
if (loginText.Text.Length > 0)
if (parolText.Text.Length > 0)
check_correct_data(loginText.Text, parolText.Text);
else
{
Err.Visibility = Visibility.Visible;
IsParol.Visibility = Visibility.Visible;
}
else
{
Err.Visibility = Visibility.Visible;
IsAll.Visibility = Visibility.Visible;
}
}
catch (Exception ex)
{
MessageBox.Show(ex.Message);
}
}
public bool check_Space(string input) => input.Any(ch =>
char.IsWhiteSpace(ch));
public bool check_SpecialChars(string input) => input.Any(ch =>
!char.IsLetterOrDigit(ch));
private void check_correct_data(string login, string password)
{
string login_from_db = "";
string query_login = "SELECT login FROM person WHERE login
= @login AND password = @password";
var param1 = new NpgsqlParameter("@login", login);
var param2 = new NpgsqlParameter("@password", password);
DataView result_login = db.ExecuteQuery(query_login, param1,
param2);
if ((result_login != null && result_login.Count > 0))
{
string dbLogin = result_login[0]["login"].ToString();
login_from_db = dbLogin;
SaveData.login = login_from_db;
}
else IsFalse.Visibility = Visibility.Visible;
string query = "SELECT password FROM person WHERE login =
@login";
var param3 = new NpgsqlParameter("@login", login);
DataView result = db.ExecuteQuery(query, param3);
if (result != null && result.Count > 0)
{
string dbPassword = result[0]["password"].ToString();
if (dbPassword == password)
{
this.Hide();
MainForm window = new MainForm();
SaveData.password = dbPassword;
getData(login_from_db);
window.Show();
window.Activate();
}
else IsFalse.Visibility = Visibility.Visible;
}
else IsFalse.Visibility = Visibility.Visible;
}
private void getData(string login)
{
string role_from_db;
string query = "SELECT rights FROM person WHERE login =
@login";
var param = new NpgsqlParameter("@login", login);
DataView result = db.ExecuteQuery(query, param);
if (result != null && result.Count > 0)
{
role_from_db = result[0]["rights"].ToString();
SaveData.role = role_from_db;

query = $"SELECT person_id FROM person WHERE login =
'{login}'";
result = db.ExecuteQuery(query);
SaveData.id = result[0]["person_id"].ToString();
if (role_from_db == "Преподаватель")
{
query = $@"SELECT sp.subject_id, s.title
FROM subject_professor sp
JOIN subject s ON sp.subject_id = s.subject_id WHERE
sp.professor_id = '{SaveData.id}'";
result = db.ExecuteQuery(query);
foreach (DataRowView row in result)
{
string title = row["title"].ToString()?.Trim();
if (!string.IsNullOrEmpty(title) &&
!SaveData.subjects.Contains(title))
SaveData.subjects.Add(title);
}
foreach (string sub in SaveData.subjects) Console.WriteLine(sub);
}
else if(role_from_db == "Студент")
{
query = $@"SELECT g.group_name FROM person p
JOIN student_group g ON p.fk_group_id = g.group_id
WHERE p.person_id = '{SaveData.id}'";
result = db.ExecuteQuery(query);
SaveData.group = result[0]["group_name"].ToString();
}
query = "SELECT TRIM(CONCAT(p.last_name, ' ', p.first_name,
' ', " +
$"COALESCE(p.patronymic, ''))) AS full_name FROM person p
WHERE person_id = '{SaveData.id}'";
result = db.ExecuteQuery(query);
SaveData.name = result[0]["full_name"].ToString();
Console.WriteLine(SaveData.id);
Console.WriteLine(SaveData.name);
Console.WriteLine(SaveData.role);
}
}
```

```
private void Viewbox_MouseLeftButtonDown(object sender,
System.Windows.Input.MouseButtonEventArgs e) =>
Close();
private void Viewbox_MouseLeftButtonDown_1(object sender,
System.Windows.Input.MouseButtonEventArgs e) =>
MessageBox.Show(@"Чтобы войти в аккаунт, введите логин и
пароль от администратора в соответствующие поля, и
нажмите кнопку 'Вход'");
private void Canvas_MouseLeftButtonDown(object sender,
System.Windows.Input.MouseButtonEventArgs e)
{
mask = !mask;
if (mask)
{
MaskPass.Visibility = Visibility.Visible;
parolText.Visibility = Visibility.Collapsed;
ClosedEye.Visibility = Visibility.Visible;
OpenEye.Visibility = Visibility.Collapsed;
}
else
{
parolText.Visibility = Visibility.Visible;
MaskPass.Visibility = Visibility.Collapsed;
ClosedEye.Visibility = Visibility.Collapsed;
OpenEye.Visibility = Visibility.Visible;
}
}
private void parolText_TextChanged(object sender,
System.Windows.Controls.TextChangedEventArgs e)
{
if(!mask) MaskPass.Password = parolText.Text;
}
private void MaskPass_PasswordChanged(object sender,
RoutedEventArgs e)
{
if (mask) parolText.Text = MaskPass.Password;
}
}
```

MainForm.xaml.cs

```
using Npgsql;
using System;
using System.Data;
using System.Linq;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Controls.Primitives;
using System.Windows.Data;
namespace WpfApp2
{
/// <summary>
/// Логика взаимодействия для Window1.xaml
/// </summary>
public partial class MainForm : Window
{
public MainForm()
{
InitializeComponent();
View.Content = new allGrades();
AllGradesButton_Click(null, null);
}

database db = new database();
public void OpenNewSub() =>
View.Content = new newSub();
private void AllGradesButton_Click(object sender,
RoutedEventArgs e)
{
if (SaveData.currentWidth == 0) this.Width = 908;
var AllGrades = new allGrades();
View.Content = AllGrades;
SetActiveButton(AllGradesButton);
}

private void DistGradeButton_Click(object sender,
RoutedEventArgs e) => OpenDist();
public void OpenDist()
{
if (SaveData.role != "Студент") this.Width = 1276;
View.Content = new Dist();
SetActiveButton(DistGradeButton);
}
private void AppointButton_Click(object sender,
RoutedEventArgs e)
{
this.Width = 908;
View.Content = new Appoint();
SetActiveButton(AppointButton);
}
private void SetActiveButton(ToggleButton button)
{
var buttons = new[] { AllGradesButton, DistGradeButton,
AppointButton, ProfileButton, AddButton, InfoButton,
PersonButton, GroupButton, SubButton };
foreach (var btn in buttons)
{
btn.IsChecked = false;
}
button.IsChecked = true;
GetDataGrid.Visibility = Visibility.Collapsed;
}
private void Window_Closed(object sender, EventArgs e) =>
Application.Current.Shutdown();
private void Button_Click(object sender, RoutedEventArgs e)
{
this.Width = 908;
OpenProfile();
SetActiveButton(ProfileButton);
}
public void OpenProfile() => View.Content = new profile();
public void OpenNewProf() => View.Content = new newProf();
private void Window_Loaded(object sender, RoutedEventArgs e)
{
Role.Text = SaveData.role;
if (SaveData.role != "Администратор")
{
AppointButton.Visibility = Visibility.Collapsed;
AddButton.Visibility = Visibility.Collapsed;
}
}
private void AddButton_Click(object sender, RoutedEventArgs e)
{
this.Width = 908;
View.Content = new Add();
SetActiveButton(AddButton);
}
private void InfoButton_Click(object sender, RoutedEventArgs e)
{
string buttonGrade = "";
string buttonDailyGrade = "в таблице указаны оценки по всем
дисциплинам у конкретного студента";
string buttonAppoint = "";
string buttonAdd = "";
if (SaveData.role != "Студент")
{
buttonGrade = "\nЧтобы изменить данные таблицы, нажмите на
кнопку 'Изменить таблицу', данные сохраняются
автоматически\n" +
"Кнопка 'Сохранить' сохраняет измененные данные";
buttonDailyGrade = @"необходимо выбрать интересующую
дисциплину с помощью 'Выберите дисциплину' и затем в
таблице покажутся оценки всех студентов по дате
Чтобы изменить данные в таблице, нажмите 'Изменить
таблицу', изменения сохраняются автоматически
Кнопка 'Сохранить' сохраняет данные";
}
if (SaveData.role == "Администратор")
{
buttonGrade += "\nКнопка 'Управление дисциплинами'
переключает страницу на 'Успеваемость'";
buttonDailyGrade += "\nКнопка 'Удалить' удаляет выбранную
дисциплину\nКнопка 'Изменить' позволяет изменить данную
дисциплину";
buttonAppoint = @"4) На странице 'Назначить', чтобы
назначить преподавателя дисциплине, необходимо выбрать
дисциплину и преподавателя с помощью полей выбора и
нажать кнопку 'Назначить'
```

```
Кнопка 'Добавить дисциплину' открывает страницу создания
дисциплины, кнопка 'Новый преподаватель' открывает
страницу создания профиля";
buttonAdd = @"5) На странице 'Добавить' указана инструкция
к созданию дисциплины или профиля
Кнопка 'Новая дисциплина' открывает страницу создания
дисциплины, кнопка 'Новый профиль' открывает страницу
создания профиля";
}
MessageBox.Show($@"Инструкция:
1) На странице 'Итоговые оценки' в таблицу выведены
итоговые оценки всех студентов группы; {buttonGrade}
2) На странице 'Успеваемость' {buttonDailyGrade}
3) Чтобы открыть профиль, нажмите на значок в правом
верхнем углу, на странице профиль указаны данные аккаунта
и кнопка 'Изменить' позволяет поменять пароль и логин
{buttonAppoint}
{buttonAdd}
");
}

private void PersonButton_Click(object sender, RoutedEventArgs
e)
{
SetActiveButton(PersonButton);
setData("person");
}

private void setData(string whichData)
{
View.Content = null;
GetDataGrid.Visibility = Visibility.Visible;

DataTable dataTable = new DataTable();

switch (whichData)
{
case "person":
dataTable = GetPersonData();
break;
case "sub":
dataTable = GetSubjectData();
break;
case "group":
dataTable = GetGroupData();
break;
default:
return;
}

GetDataGrid.ItemsSource = dataTable.DefaultView;
GetDataGrid.IsReadOnly = true;
}

private DataTable GetPersonData()
{
string query = @"
SELECT
TRIM(CONCAT(p.last_name, ' ', p.first_name, ' ',
COALESCE(p.patronymic, ''))) AS ""ФИО"",
p.rights AS ""Тип учетной записи"",
p.person_id AS ""ID"",
p.login AS ""Логин"",
g.group_name AS ""Группа""
FROM person p
LEFT JOIN student_group g ON p.fk_group_id = g.group_id
ORDER BY p.rights, p.last_name, p.first_name";

DataTable dt = db.DataQuery(query);

DataTable resultTable = new DataTable();
resultTable.Columns.Add("ФИО", typeof(string));
resultTable.Columns.Add("Тип учетной записи", typeof(string));

foreach (DataRow row in dt.Rows)
{
DataRow newRow = resultTable.NewRow();
newRow["ФИО"] = row["ФИО"];
newRow["Тип учетной записи"] = row["Тип учетной записи"];
resultTable.Rows.Add(newRow);
}

return resultTable;
}

private DataTable GetSubjectData()
{
string subjectsWithProfessorsQuery = @"
SELECT
s.title AS ""Дисциплина"",
STRING_AGG(
TRIM(CONCAT(p.last_name, ' ', p.first_name, ' ',
COALESCE(p.patronymic, ''))),
', '
) AS ""Преподаватели"",
s.subject_id AS ""ID""
FROM subject s
LEFT JOIN subject_professor sp ON s.subject_id = sp.subject_id
LEFT JOIN person p ON sp.professor_id = p.person_id
GROUP BY s.subject_id, s.title
ORDER BY s.title";

DataTable subjectsDt =
db.DataQuery(subjectsWithProfessorsQuery);

string subjectsWithGroupsQuery = @"
SELECT
s.title AS ""Дисциплина"",
STRING_AGG(g.group_name, ', ') AS ""Группы""
FROM subject s
LEFT JOIN subject_group sg ON s.subject_id = sg.subject_id
LEFT JOIN student_group g ON sg.group_id = g.group_id
GROUP BY s.subject_id, s.title
ORDER BY s.title";

DataTable groupsDt = db.DataQuery(subjectsWithGroupsQuery);

DataTable resultTable = new DataTable();
resultTable.Columns.Add("Дисциплина", typeof(string));
resultTable.Columns.Add("Преподаватели", typeof(string));
resultTable.Columns.Add("Группы", typeof(string));

foreach (DataRow subjectRow in subjectsDt.Rows)
{
string subjectName = subjectRow["Дисциплина"].ToString();
string professors = subjectRow["Преподаватели"].ToString();

string groups = "";
var groupRows = groupsDt.Select($"Дисциплина =
'{subjectName.Replace("'", "'")}'");
if (groupRows.Length > 0)
{
groups = groupRows[0]["Группы"].ToString();
}

DataRow newRow = resultTable.NewRow();
newRow["Дисциплина"] = subjectName;
newRow["Преподаватели"] = string.IsNullOrEmpty(professors) ?
"Нет преподавателей" : professors;
newRow["Группы"] = string.IsNullOrEmpty(groups) ? "Нет
групп" : groups;
resultTable.Rows.Add(newRow);
}

return resultTable;
}

private DataTable GetGroupData()
{
string groupsWithSubjectsQuery = @"
SELECT
g.group_name AS ""Группа"",
STRING_AGG(DISTINCT s.title, ', ') AS ""Дисциплины"",
g.group_id AS ""ID""
FROM student_group g
LEFT JOIN subject_group sg ON g.group_id = sg.group_id
LEFT JOIN subject s ON sg.subject_id = s.subject_id
GROUP BY g.group_id, g.group_name
```

```
ORDER BY g.group_name";

DataTable subjectsDt = db.DataQuery(groupsWithSubjectsQuery);

string groupsWithStudentsQuery = @"
SELECT
g.group_name AS ""Группа"",
STRING_AGG(
TRIM(CONCAT(p.last_name, ' ', p.first_name, ' ',
COALESCE(p.patronymic, ''))),
', '
) AS ""Студенты""
FROM student_group g
LEFT JOIN person p ON g.group_id = p.fk_group_id AND
p.rights = 'Студент'
GROUP BY g.group_id, g.group_name
ORDER BY g.group_name";

DataTable studentsDt = db.DataQuery(groupsWithStudentsQuery);

DataTable resultTable = new DataTable();
resultTable.Columns.Add("Группа", typeof(string));
resultTable.Columns.Add("Дисциплины", typeof(string));
resultTable.Columns.Add("Студенты", typeof(string));

foreach (DataRow subjectRow in subjectsDt.Rows)
{
string groupName = subjectRow["Группа"].ToString();
string subjects = subjectRow["Дисциплины"].ToString();

string students = "";
var studentRows = studentsDt.Select($"Группа =
'{groupName.Replace("", """)}'");
if (studentRows.Length > 0)
{
students = studentRows[0]["Студенты"].ToString();
}

DataRow newRow = resultTable.NewRow();
newRow["Группа"] = groupName;
newRow["Дисциплины"] = string.IsNullOrEmpty(subjects) ?
"Нет дисциплин" : subjects;
newRow["Студенты"] = string.IsNullOrEmpty(students) ? "Нет
студентов" : students;
resultTable.Rows.Add(newRow);
}

return resultTable;
}

private void SubButton_Click(object sender, RoutedEventArgs e)
{
SetActiveButton(SubButton);
setData("sub");
}

private void GroupButton_Click(object sender, RoutedEventArgs
e)
{
SetActiveButton(GroupButton);
setData("group");
}
}
}
```

Add.xaml.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;
namespace WpfApp2
{
/// <summary>
/// Логика взаимодействия для Add.xaml
/// </summary>
public partial class Add : Page
{
public Add()
{
InitializeComponent();
}
private void AddDist_Click(object sender, RoutedEventArgs e)
{
SaveData.isChange = false;
((MainForm)Window.GetWindow(this)).OpenNewSub();
}
private void AddProf_Click(object sender, RoutedEventArgs e) =>
((MainForm)Window.GetWindow(this)).OpenNewProf();
}
}
```

AllGrades.xaml.cs

```
using Npgsql;
using Syncfusion.Windows.Shared;
using System;
using System.Collections.Generic;
using System.Data;
using System.Globalization;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Markup;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;
namespace WpfApp2
{
/// <summary>
/// Логика взаимодействия для Page1.xaml
/// </summary>
///
public class OffsetToBoolConverter : IValueConverter
{
public object Convert(object value, Type targetType, object
parameter, CultureInfo culture)
{
return value?.ToString().Trim() == "3";
}
public object ConvertBack(object value, Type targetType, object
parameter, CultureInfo culture)
{
if (value is bool boolVal && boolVal)
return "3";
return null;
}
}
public partial class allGrades : Page
{
public allGrades()
{
InitializeComponent();
}
private bool isChange = true;
database db = new database();
```

```
public void SetData(DataTable dt, List<string> offsetColumns)
{
this.offsetColumns = offsetColumns;

Grades.ItemsSource = null;
Grades.Columns.Clear();

if (dt != null && dt.Rows.Count > 0)
{
Grades.ItemsSource = dt.DefaultView;
Grades.AutoGenerateColumns = true;
}
else
{
DataTable emptyTable = new DataTable();
emptyTable.Columns.Add("Студент", typeof(string));
Grades.ItemsSource = emptyTable.DefaultView;
Grades.AutoGenerateColumns = true;
}
}
private List<string> offsetColumns = new List<string>();
private void Grades_AutoGeneratingColumn(object sender,
DataGridAutoGeneratingColumnEventArgs e)
{
string columnHeader = e.Column.Header.ToString();
if (SaveData.role == "Преподаватель" && SaveData.subjects !=
null && SaveData.subjects.Any())
{
if (columnHeader != "Студент")
{
bool isTeacherSubject = SaveData.subjects.Any(sub =>
!string.IsNullOrEmpty(sub) &&
columnHeader.Contains(sub.Trim()));
if (!isTeacherSubject)
{
e.Cancel = true;
return;
}
}
}
if (columnHeader == "Студент")
{
e.Column.IsReadOnly = true;
return;
}
if (offsetColumns.Contains(columnHeader))
{
DataGridCheckBoxColumn checkBoxColumn = new
DataGridCheckBoxColumn
{
Header = columnHeader,
Binding = new Binding(columnHeader)
{
Converter = new OffsetToBoolConverter(),
UpdateSourceTrigger = UpdateSourceTrigger.PropertyChanged
}
};
e.Column = checkBoxColumn;
}
}
private void IsChange_Click(object sender, RoutedEventArgs e)
=>
Grades.IsReadOnly = !Grades.IsReadOnly;
private void SaveButton_Click(object sender, RoutedEventArgs e)
=>
MessageBox.Show("Данные сохранены");
private void Grades_CellEditEnding(object sender,
DataGridCellEditEndingEventArgs e)
{
if (e.EditAction != DataGridEditAction.Commit) return;
e.Cancel = true;
var column = e.Column as DataGridBoundColumn;
if (column == null || column.IsReadOnly) return;
var bindingPath = (column.Binding as Binding)?.Path.Path;
if (bindingPath == "Студент") return;
var row = e.Row.Item as DataRowView;
if (row == null) return;
object oldValue = row[bindingPath];
string oldGrade = oldValue?.ToString();
string newGrade = null;
bool isValidInput = true;
if (e.EditingElement is TextBox textBox)
{
newGrade = textBox.Text.Trim();
if (string.IsNullOrEmpty(newGrade)) newGrade = null;
if (newGrade != null && (!int.TryParse(newGrade, out int grade) ||
grade < 2 || grade > 5))
{
MessageBox.Show("Ошибка: Оценка может быть только от 2
до 5");
isValidInput = false;
textBox.Text = oldGrade ?? string.Empty;
textBox.SelectAll();
textBox.Focus();
row.Row.SetField(bindingPath, oldValue);
return;
}
}
else if (e.EditingElement is CheckBox checkBox)
{
newGrade = checkBox.IsChecked == true ? "3" : null;
}
else
{
return;
}
if (isValidInput)
{
string studentName = row["Студент"].ToString();
SaveGradeToDatabase(studentName, bindingPath, newGrade);
}
}
public void SaveGradeToDatabase(string studentName, string
subjectTitle, string newGrade)
{
bool isEmpty = string.IsNullOrWhiteSpace(newGrade);
string[] parts = studentName.Split(' ');
if (parts.Length < 2) return;
string lastName = parts[0];
string firstName = parts[1];
string patronymic = parts.Length > 2 ? parts[2] : null;
bool isOffset = false;
string isOffsetQuery = $@"
SELECT isoffset
FROM subject
WHERE title = '{subjectTitle}'";
var result = db.ExecuteQuery(isOffsetQuery);
if (result.Count > 0 && result[0]["isoffset"] != null)
{
isOffset = bool.Parse(result[0]["isoffset"].ToString());
}
else
{
MessageBox.Show($"Ошибка: Предмет '{subjectTitle}' не
найден в базе данных");
return;
}
object noteValue = DBNull.Value;
bool offsetValue = false;
if (!isEmpty)
{
if (isOffset)
{
offsetValue = (newGrade.Trim() == "3");
}
else
{
if (!int.TryParse(newGrade, out int grade) || grade < 2 || grade > 5)
{
MessageBox.Show("Ошибка: Оценка может быть только от 2
до 5");
return;
}
noteValue = grade;
offsetValue = false;
}
}
var parameters = new List<NpgsqlParameter>
```

```
{
new NpgsqlParameter("title", subjectTitle),
new NpgsqlParameter("last_name", lastName),
new NpgsqlParameter("first_name", firstName),
new NpgsqlParameter("patronymic", (object)patronymic ??
DBNull.Value)
};
string sql;
if (isEmpty || (isOffset && !offsetValue))
{
sql = @"
DELETE FROM notes
USING person p, subject s
WHERE notes.fk_person_id = p.person_id
  AND notes.fk_subject_id = s.subject_id
  AND p.last_name = @last_name
  AND p.first_name = @first_name
  AND (p.patronymic = @patronymic OR (p.patronymic IS NULL
AND @patronymic IS NULL))
  AND s.title = @title";
}
else
{
sql = @"
INSERT INTO notes (fk_person_id, fk_subject_id, note, isoffset)
SELECT p.person_id, s.subject_id, @note, @offsetValue
FROM person p
JOIN subject s ON s.title = @title
WHERE p.first_name = @first_name
  AND p.last_name = @last_name
  AND (p.patronymic = @patronymic OR (p.patronymic IS NULL
AND @patronymic IS NULL))
ON CONFLICT (fk_person_id, fk_subject_id)
DO UPDATE SET
note = EXCLUDED.note,
isoffset = EXCLUDED.isoffset";
parameters.Add(new NpgsqlParameter("note", noteValue));
parameters.Add(new NpgsqlParameter("offsetValue",
offsetValue));
}
db.ExecuteNonQuery(sql, parameters.ToArray());
}
private void Page_Loaded(object sender, RoutedEventArgs e)
{
if (SaveData.role == "Студент")
{
ButtonsStack.Visibility = Visibility.Collapsed;
ChangeStack.Visibility = Visibility.Collapsed;
string query = $"SELECT fk_group_id FROM person WHERE
person_id = {SaveData.id}";
DataView result = db.ExecuteQuery(query);
string group_id = result[0]["fk_group_id"].ToString();
query = $"SELECT group_name FROM student_group WHERE
group_id = '{group_id}'";
result = db.ExecuteQuery(query);
SaveData.groups.Add(result[0]["group_name"].ToString());
Groups.ItemsSource = SaveData.groups;
Groups.SelectedIndex = 0;
return;
}
else if (SaveData.role == "Преподаватель")
ManageButton.Visibility = Visibility.Collapsed;
string groups = "SELECT group_name FROM student_group";
DataView res = db.ExecuteQuery(groups);
foreach (DataRowView row in res)
{
string title = row["group_name"].ToString()?.Trim();
if (!string.IsNullOrEmpty(title) &&
!SaveData.groups.Contains(title))
SaveData.groups.Add(title);
}
Groups.ItemsSource = SaveData.groups;
Groups.SelectedIndex = 0;
}
private void ManageButton_Click(object sender, RoutedEventArgs
e)
{
((MainForm)Window.GetWindow(this)).OpenDist();
}

private void Groups_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
if (Groups.SelectedItem == null) return;

string group_name = Groups.SelectedItem.ToString();
string getIdQuery = $"SELECT group_id FROM student_group
WHERE group_name = '{group_name}'";
DataView idResult = db.ExecuteQuery(getIdQuery);
string group_id = idResult[0]["group_id"].ToString();

// ИЗМЕНЕНИЕ: Теперь получаем предметы через JOIN с
subject_group
string subjectsQuery = $@"
SELECT s.subject_id, s.title, s.isoffset
FROM subject s
JOIN subject_group sg ON s.subject_id = sg.subject_id
WHERE sg.group_id = '{group_id}'
ORDER BY s.title";

DataTable subjectsTable = db.DataQuery(subjectsQuery);
var titles = subjectsTable.AsEnumerable()
.Select(row => new {
Title = row["title"].ToString(),
IsOffset = Convert.ToBoolean(row["isoffset"])
}).ToList();

var pivotColumns = string.Join(",\n", titles.Select(s =>
{
if (s.IsOffset)
{
return $@"MAX(CASE WHEN subj.title = '{s.Title}' AND
n.isoffset = true THEN '3' ELSE NULL END) AS ""{s.Title}""";
}
else
{
return $@"MAX(CASE WHEN subj.title = '{s.Title}' THEN
n.note ELSE NULL END) AS ""{s.Title}""";
}
}));

string Command = $@"
SELECT
TRIM(CONCAT(p.last_name, '', p.first_name, '',
COALESCE(p.patronymic, ''))) AS ""Студент"",
{pivotColumns}
FROM person p
LEFT JOIN notes n ON n.fk_person_id = p.person_id
LEFT JOIN subject subj ON subj.subject_id = n.fk_subject_id
WHERE p.rights = 'Студент'
AND p.fk_group_id = '{group_id}'
GROUP BY p.person_id, p.last_name, p.first_name, p.patronymic
ORDER BY p.last_name, p.first_name";

DataTable dt = db.DataQuery(Command);
SetData(dt, titles.Where(t => t.IsOffset).Select(t =>
t.Title).ToList());
}
}
}
```

Appoint.xaml.cs

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Windows;
using System.Windows.Controls;
namespace WpfApp2
{
public partial class Appoint : Page
{
public Appoint()
{
InitializeComponent();
}
```

```
database db = new database();
private void AddDist_Click(object sender, RoutedEventArgs e)
{
SaveData.isChange = false;
((MainForm)Window.GetWindow(this)).OpenNewSub();
}
private void Page_Loaded(object sender, RoutedEventArgs e)
{
List<string> subjects = new List<string>();
string query = "SELECT title FROM subject ORDER BY title";
var res = db.ExecuteQuery(query);
foreach (DataRowView row in res)
{
string title = row["title"].ToString()?.Trim();
if (!string.IsNullOrEmpty(title) && !subjects.Contains(title))
subjects.Add(title);
}
Subjects.ItemsSource = subjects;
query = @"SELECT TRIM(CONCAT(p.last_name, '',
p.first_name, '', COALESCE(p.patronymic, '''))) AS ""prof""
 FROM person p
 WHERE rights = 'Преподаватель'
 ORDER BY p.last_name, p.first_name";
var profsDB = db.ExecuteQuery(query);
var profs = profsDB.Cast<DataRowView>().Select(row =>
row["prof"].ToString()).ToList();
Profs.ItemsSource = profs;
}
private void AppointButton_Click(object sender,
RoutedEventArgs e)
{
if (Subjects.SelectedItem == null || Profs.SelectedItem == null)
{
MessageBox.Show("Выберите дисциплину и преподавателя");
return;
}
var sub = Subjects.SelectedItem.ToString();
var prof = Profs.SelectedItem.ToString();
string[] parts = prof.Split(new[] { ' ' },
StringSplitOptions.RemoveEmptyEntries);
if (parts.Length < 2)
{
MessageBox.Show("Некорректное ФИО преподавателя");
return;
}
string lastName = parts[0];
string firstName = parts[1];
string patronymic = parts.Length > 2 ? parts[2] : null;
try
{
string query = $@"
WITH subject_cte AS (
SELECT subject_id FROM subject WHERE title = '{sub}'
),
professor_cte AS (
SELECT person_id
FROM person
WHERE first_name = '{firstName}'
  AND last_name = '{lastName}'";
if (patronymic != null)
{
query += $" AND patronymic = '{patronymic}'";
}
else
{
query += " AND patronymic IS NULL";
}
query += @"
)
INSERT INTO subject_professor(subject_id, professor_id)
SELECT s.subject_id, p.person_id
FROM subject_cte s, professor_cte p
WHERE EXISTS (SELECT 1 FROM subject_cte)
  AND EXISTS (SELECT 1 FROM professor_cte)
ON CONFLICT (subject_id, professor_id) DO NOTHING
RETURNING subject_id";
var result = db.ExecuteQuery(query);
if (result != null && result.Count > 0)
{
MessageBox.Show("Преподаватель успешно назначен на
дисциплину");
}
else
{
string checkQuery = $@"
SELECT COUNT(*)
FROM subject s, person p
WHERE s.title = '{sub}'
  AND p.first_name = '{firstName}'
  AND p.last_name = '{lastName}'";
if (patronymic != null)
{
checkQuery += $" AND p.patronymic = '{patronymic}'";
}
else
{
checkQuery += " AND p.patronymic IS NULL";
}
var checkResult = db.ExecuteQuery(checkQuery);
if (checkResult != null && checkResult.Count > 0)
{
int subjectCount = Convert.ToInt32(checkResult[0][0]);
if (subjectCount == 0)
{
MessageBox.Show("Предмет или преподаватель не найден");
}
else
{
MessageBox.Show("Этот преподаватель уже ведет эту
дисциплину");
}
}
else
{
MessageBox.Show("Не удалось проверить данные");
}
}
}
catch (Exception ex)
{
MessageBox.Show($"Ошибка при назначении преподавателя:
{ex.Message}");
}
}
private void AddProf_Click(object sender, RoutedEventArgs e) =>
((MainForm)Window.GetWindow(this)).OpenNewProf();
}
}
```

Dist.xaml.cs

```
using Npgsql;
using System;
using System.Collections.Generic;
using System.Data;
using System.Globalization;
using System.Linq;
using System.Text.RegularExpressions;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Input;

namespace WpfApp2
{
public partial class Dist : Page
{
public Dist()
{
InitializeComponent();
}

private bool isChange = false;
database db = new database();
string group_id;
public void SetData(DataTable dt)
{
```

```
DistGrades.ItemsSource = dt.DefaultView;
}

private DataTable AddAverageColumn(DataTable dt)
{
if (dt.Columns.Count == 0 || dt.Rows.Count == 0)
return dt;

DataTable newTable = dt.Clone();
newTable.Columns.Add("Средняя оценка", typeof(string));

foreach (DataRow row in dt.Rows)
{
DataRow newRow = newTable.NewRow();

for (int i = 0; i < dt.Columns.Count; i++)
{
newRow[i] = row[i];
}

double sum = 0;
int count = 0;

foreach (DataColumn col in dt.Columns)
{
if (col.ColumnName == "Студент")
continue;

object cellValue = row[col];
if (cellValue != null && cellValue != DBNull.Value)
{
if (double.TryParse(cellValue.ToString(), out double grade))
{
sum += grade;
count++;
}
}
}

if (count > 0)
{
double average = sum / count;
newRow["Средняя оценка"] = average.ToString("F2");
}
else
{
newRow["Средняя оценка"] = "—";
}

newTable.Rows.Add(newRow);
}

return newTable;
}

private void Page_Loaded(object sender, RoutedEventArgs e)
{
if (SaveData.role == "Студент")
{
ChoiceStack.Visibility = Visibility.Collapsed;
DistGrades.Margin = new Thickness(-300, 24, 20, 30);
AddData.Visibility = Visibility.Collapsed;
LoadData();
return;
}

Groups.ItemsSource = SaveData.groups;
Groups.SelectedIndex = 0;
List<string> subjects = new List<string>();
Dists.ItemsSource = subjects;
if (SaveData.currentSub != null) Dists.SelectedItem =
SaveData.currentSub;

if (SaveData.role == "Преподаватель")
{
butStack.Visibility = Visibility.Collapsed;
foreach (string sub in SaveData.subjects)
if (!string.IsNullOrEmpty(sub)) subjects.Add(sub);
}
else
{
string group_name = Groups.SelectedItem?.ToString();
if (!string.IsNullOrEmpty(group_name))
{
string getIdQuery = $"SELECT group_id FROM student_group
WHERE group_name = '{group_name}'";
DataView idResult = db.ExecuteQuery(getIdQuery);
if (idResult != null && idResult.Count > 0)
{
string current_group_id = idResult[0]["group_id"].ToString();

string query = $@"
SELECT s.title
FROM subject s
JOIN subject_group sg ON s.subject_id = sg.subject_id
WHERE sg.group_id = '{current_group_id}'
ORDER BY s.title";

var res = db.ExecuteQuery(query);

foreach (DataRowView row in res)
{
string title = row["title"].ToString()?.Trim();
if (!string.IsNullOrEmpty(title) && !subjects.Contains(title))
subjects.Add(title);
}
}
}
}
}

private void Save_Click(object sender, RoutedEventArgs e) =>
MessageBox.Show("Данные сохранены");

private void IsChange_Click(object sender, RoutedEventArgs e)
=>
DistGrades.IsReadOnly = !DistGrades.IsReadOnly;

private void DistGrades_CellEditEnding(object sender,
DataGridCellEditEndingEventArgs e)
{
if (e.EditAction == DataGridEditAction.Commit)
{
var column = e.Column as DataGridBoundColumn;
if (column != null)
{
var bindingPath = (column.Binding as Binding)?.Path.Path;
Console.WriteLine(bindingPath);
var editedValue = (e.EditingElement as TextBox)?.Text;
var subject = Dists.SelectedItem.ToString();
var row = e.Row.Item as DataRowView;

if (string.IsNullOrEmpty(editedValue)) editedValue = null;
else if (!int.TryParse(editedValue, out _)) return;

if (editedValue != null)
if (int.Parse(editedValue) > 5 || int.Parse(editedValue) < 2)
{
MessageBox.Show("Ошибка: Оценка может быть только от 2
до 5");
(e.EditingElement as TextBox).Text = null;
return;
}

if (row != null && bindingPath != "Студент" && bindingPath !=
"Средняя оценка")
{
string studentName = row["Студент"].ToString();
SaveGradeToDatabase(studentName, bindingPath, subject,
editedValue);

Dispatcher.BeginInvoke(new Action(() =>
{
if (Dists.SelectedItem != null)
getData(Dists.SelectedItem.ToString());
}), System.Windows.Threading.DispatcherPriority.Background);
}
}
```

```
}
}

public void LoadData()
{
string person_id = SaveData.id;

string studentGroupQuery = $"SELECT fk_group_id FROM
person WHERE person_id = '{person_id}'";
DataView groupResult = db.ExecuteQuery(studentGroupQuery);
string student_group_id =
groupResult[0]["fk_group_id"].ToString();

string datesSql = $@"
SELECT DISTINCT d.datetime::date AS date
FROM daily_notes d
JOIN subject s ON d.fk_subject_id = s.subject_id
JOIN subject_group sg ON s.subject_id = sg.subject_id
WHERE d.fk_person_id = '{person_id}'
AND sg.group_id = '{student_group_id}'
ORDER BY date";

var datesResult = db.ExecuteQuery(datesSql);
var dates = datesResult?.Cast<DataRowView>()
.Select(r => ((DateTime)r["date"]).ToString("yyyy-MM-dd"))
.ToList() ?? new List<string>();

var pivotColumns = string.Join(",\n", dates.Select(d =>
$@"MAX(CASE WHEN d.datetime::date = DATE '{d}' THEN
d.note ELSE NULL END) AS ""{d}"""));

var columnsToUse = string.IsNullOrEmpty(pivotColumns) ?
"NULL AS 'Нет данных'" : pivotColumns;

string query = $@"
SELECT
s.title AS ""Дисциплины"",
{columnsToUse}
FROM subject s
JOIN subject_group sg ON s.subject_id = sg.subject_id
LEFT JOIN daily_notes d
ON d.fk_person_id = '{person_id}'
  AND d.fk_subject_id = s.subject_id
WHERE sg.group_id = '{student_group_id}'
GROUP BY s.subject_id, s.title
ORDER BY ""Дисциплины"";

try
{
DataTable dt = db.DataQuery(query);
DataTable dtWithAverage = AddAverageColumn(dt);
SetData(dtWithAverage);
}
catch (Exception ex)
{
MessageBox.Show(ex.Message);
}
}

public void SaveGradeToDatabase(string studentName, string
date, string subjectTitle, string newGrade)
{
try
{
if (newGrade == null) newGrade = "NULL";

string[] parts = studentName.Split(' ');
if (parts.Length < 2) return;

string lastName = parts[0];
string firstName = parts[1];
string patronymic = parts.Length > 2 ? parts[2] : null;

string dbDate = date;

if (newGrade == "NULL")
{
string sql = $@"
DELETE FROM daily_notes dn
USING person p, subject s
WHERE dn.fk_person_id = p.person_id
  AND dn.fk_subject_id = s.subject_id
  AND p.last_name = '{lastName}'
  AND p.first_name = '{firstName}'
  AND (p.patronymic = '{patronymic}' OR (p.patronymic IS
NULL AND '{patronymic}' IS NULL))
  AND s.title = '{subjectTitle}'
  AND dn.datetime = '{dbDate}'";

db.ExecuteNonQuery(sql);
}
else
{
string sql = $@"
INSERT INTO daily_notes (datetime, fk_person_id, fk_subject_id,
note)
SELECT '{dbDate}', p.person_id, s.subject_id, {newGrade}
FROM person p
JOIN subject s ON s.title = '{subjectTitle}'
WHERE p.first_name = '{firstName}'
AND p.last_name = '{lastName}'
AND (p.patronymic = @patronymic OR (p.patronymic IS NULL
AND @patronymic IS NULL))
ON CONFLICT (fk_person_id, fk_subject_id, datetime)
DO UPDATE SET note = EXCLUDED.note";

var parameter = new NpgsqlParameter("@patronymic",
(object)patronymic ?? DBNull.Value);
db.ExecuteNonQuery(sql, parameter);
}
}
catch (Exception ex)
{
MessageBox.Show($"Ошибка сохранения оценки:
{ex.Message}");
}
}

private void getData(string subject)
{
try
{
SaveData.currentSub = subject;

var id_res = db.ExecuteQuery($"SELECT subject_id FROM
subject WHERE title = '{subject}'");
if (id_res?.Count == 0)
{
MessageBox.Show("Предмет не найден!");
return;
}

int subject_id = Convert.ToInt32(id_res[0]["subject_id"]);

string checkSubjectGroupQuery = $@"
SELECT COUNT(*) as count
FROM subject_group
WHERE subject_id = {subject_id}
AND group_id = '{group_id}'";

DataView checkResult =
db.ExecuteQuery(checkSubjectGroupQuery);
if (checkResult != null && checkResult.Count > 0 &&
Convert.ToInt32(checkResult[0]["count"]) == 0)
{
MessageBox.Show("Данный предмет не относится к
выбранной группе!");
return;
}

string datesSql = $@"
SELECT DISTINCT datetime::date AS date
FROM daily_notes
WHERE fk_subject_id = {subject_id}
ORDER BY date";

var datesResult = db.ExecuteQuery(datesSql);
```

```
if (datesResult == null || datesResult.Count == 0)
{
MessageBox.Show("Нет оценок по данной дисциплине!");
DataTable emptyTable = new DataTable();
emptyTable.Columns.Add("Студент", typeof(string));
emptyTable.Columns.Add("Средняя оценка", typeof(string));
DataRow row = emptyTable.NewRow();
row["Студент"] = "Оценок нет";
row["Средняя оценка"] = "—";
emptyTable.Rows.Add(row);
SetData(emptyTable);
return;
}

var dates = datesResult.Cast<DataRowView>()
.Select(r => ((DateTime)r["date"]).ToString("yyyy-MM-dd"))
.ToList();

var pivotColumns = string.Join(",\n", dates.Select(d =>
$@"MAX(CASE WHEN d.datetime::date = DATE '{d}' THEN
d.note ELSE NULL END) AS ""{d}"""));

var columnsToUse = string.IsNullOrEmpty(pivotColumns) ?
"NULL AS 'Нет данных'" : pivotColumns;

string query = $@"
SELECT
TRIM(CONCAT(p.last_name, ' ', p.first_name, ' ',
COALESCE(p.patronymic, ''))) AS ""Студент"",
{columnsToUse}
FROM person p
LEFT JOIN daily_notes d
ON d.fk_person_id = p.person_id
   AND d.fk_subject_id = {subject_id}
WHERE p.rights = 'Студент' AND p.fk_group_id = '{group_id}'
GROUP BY p.person_id, p.last_name, p.first_name, p.patronymic
ORDER BY ""Студент"";

DataTable dt = db.DataQuery(query);

DataTable dtWithAverage = AddAverageColumn(dt);
SetData(dtWithAverage);
}
catch (Exception ex)
{
MessageBox.Show(ex.Message);
}
}

private void DistGrades_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
var sub = Dists.SelectedItem;
if (sub != null)
{
getData(sub.ToString());
}
}

private void Button_Click_1(object sender, RoutedEventArgs e)
=>
DataStack.Visibility = Visibility.Visible;

private void InputData_TextChanged(object sender,
TextChangedEventArgs e)
{
var textBox = (TextBox)sender;
var text = textBox.Text;

text = Regex.Replace(text, @"[^0-9]", "");

if (text.Length >= 2)
{
text = text.Insert(2, "-");
}
if (text.Length >= 5)
{
text = text.Insert(5, "-");
}

if (text.Length > 10)
{
text = text.Substring(0, 10);
}

textBox.Text = text;
textBox.SelectionStart = textBox.Text.Length;
}

private void InputData_PreviewTextInput(object sender,
TextCompositionEventArgs e) =>
e.Handled = !Regex.IsMatch(e.Text, @"[0-9]+$");

private void InputData_LostFocus(object sender,
RoutedEventArgs e)
{
string newData = InputData.Text;

if (!DateTime.TryParseExact(newData, "dd-MM-yyyy",
CultureInfo.InvariantCulture, DateTimeStyles.None, out _))
{
MessageBox.Show("Некорректная дата! День: 01-31, Месяц:
01-12.");
InputData.Clear();
}

string checkData = $@"SELECT datetime
  FROM daily_notes
  WHERE datetime = '{newData}'";

DataView result = db.ExecuteQuery(checkData);

if (result == null || result.Count <= 1)
{
string query = $@"INSERT INTO daily_notes (datetime)
 VALUES('{newData}')";

db.ExecuteNonQuery(query);
SaveGradeToDatabase("Новикова Мария Алексеевна", newData,
Dists.SelectedItem.ToString(), "5");

DataStack.Visibility = Visibility.Collapsed;

if (Dists.SelectedItem != null)
getData(Dists.SelectedItem.ToString());
}
else
MessageBox.Show("Запись этой даты уже существует, введите
оценку в колонку");
}

private void Delete_Click(object sender, RoutedEventArgs e)
{
try
{
if (Dists.SelectedItem != null)
{
var subject = Dists.SelectedItem.ToString();
MessageBoxResult result;

string checkDependencies = @"
SELECT COUNT(*) as count FROM daily_notes dn
JOIN subject s ON dn.fk_subject_id = s.subject_id
WHERE s.title = @subject

UNION ALL

SELECT COUNT(*) as count FROM subject_professor sp
JOIN subject s ON sp.subject_id = s.subject_id
WHERE s.title = @subject

UNION ALL

SELECT COUNT(*) as count FROM subject_group sg
JOIN subject s ON sg.subject_id = s.subject_id
WHERE s.title = @subject";
```

```
var checkParams = new[] { new NpgsqlParameter("@subject",
subject) };
var dependencyResult = db.ExecuteQuery(checkDependencies,
checkParams);

bool hasDependencies = false;
foreach (DataRowView row in dependencyResult)
{
if (Convert.ToInt32(row["count"]) > 0)
{
hasDependencies = true;
break;
}
}

if (hasDependencies)
{
result = MessageBox.Show(
$"У дисциплины '{subject}' есть связанные записи (оценки,
преподаватели или группы). " +
"Удалить всё? Это действие нельзя отменить.",
"Предупреждение о зависимостях",
MessageBoxButton.YesNo,
MessageBoxImage.Warning);

if (result != MessageBoxResult.Yes)
{
return;
}
}

result = MessageBox.Show(
$"Вы уверены, что хотите удалить дисциплину {subject}?",
"Подтверждение",
MessageBoxButton.YesNo,
MessageBoxImage.Question);

if (result == MessageBoxResult.Yes)
{
string query = $@"DELETE FROM subject
WHERE title = '{subject}'";

db.ExecuteNonQuery(query);
MessageBox.Show($"Дисциплина {subject} успешно удалена");

Dists.SelectedItem = null;
SaveData.currentSub = null;

Page_Loaded(sender, e);
}
}
else
MessageBox.Show("Выберите дисциплину для удаления!");
}
catch (Exception ex)
{
MessageBox.Show(ex.Message);
}
}

private void Change_Click(object sender, RoutedEventArgs e)
{
if (Dists.SelectedItem != null)
{
SaveData.isChange = true;
((MainForm)Window.GetWindow(this)).OpenNewSub();
}
else MessageBox.Show("Выберите дисциплину для
изменения");
}

private void Groups_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
if (Groups.SelectedItem == null) return;

string group_name = Groups.SelectedItem.ToString();
string getIdQuery = $"SELECT group_id FROM student_group
WHERE group_name = '{group_name}'";
DataView idResult = db.ExecuteQuery(getIdQuery);
group_id = idResult[0]["group_id"].ToString();

List<string> subjects = new List<string>();

if (SaveData.role == "Преподаватель")
{
foreach (string subject in SaveData.subjects)
{
if (!string.IsNullOrEmpty(subject))
{
string checkQuery = $@"
SELECT COUNT(*) as count
FROM subject s
JOIN subject_group sg ON s.subject_id = sg.subject_id
WHERE s.title = '{subject}'
AND sg.group_id = '{group_id}'";

DataView checkResult = db.ExecuteQuery(checkQuery);
if (checkResult != null && checkResult.Count > 0 &&
Convert.ToInt32(checkResult[0]["count"]) > 0)
{
subjects.Add(subject);
}
}
}
}
else
{
string query = $@"
SELECT s.title
FROM subject s
JOIN subject_group sg ON s.subject_id = sg.subject_id
WHERE sg.group_id = '{group_id}'
ORDER BY s.title";

var res = db.ExecuteQuery(query);

foreach (DataRowView row in res)
{
string title = row["title"].ToString()?.Trim();
if (!string.IsNullOrEmpty(title) && !subjects.Contains(title))
subjects.Add(title);
}
}

Dists.ItemsSource = subjects;
}
}
}
```

newProf.xaml.cs

```
using Npgsql;
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text.RegularExpressions;
using System.Windows;
using System.Windows.Controls;

namespace WpfApp2
{
public partial class newProf : Page
{
public newProf()
{
InitializeComponent();
}

database db = new database();
List<string> subjects = new List<string>();

private void Save_Click(object sender, RoutedEventArgs e)
{
if (CheckError())
{
return;
```

```
}

string name = Name.Text.Trim();
string lastName = LastName.Text.Trim();
string patronymic = Patronymic.Text?.Trim();
string login = Login.Text.Trim();
string password = Password.Text.Trim();
string types = Types.SelectedItem.ToString();

if (IsPersonExists(name, lastName, patronymic))
{
MessageBox.Show($"Пользователь {lastName} {name}
{patronymic} уже существует в базе данных!",
"Ошибка", MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}

string message = $"Создать учетную запись {types.ToLower()}а
{lastName} {name} {patronymic}?";

if (types == "Студент" && Group.SelectedItem != null)
{
message += $"\nГруппа: {Group.SelectedItem}";
}

if (types == "Преподаватель" && subjects.Count > 0)
{
message += $"\nДисциплины: {string.Join(", ", subjects)}";
}

MessageBoxResult result = MessageBox.Show(
message,
"Подтверждение",
MessageBoxButton.YesNo,
MessageBoxImage.Question);

if (result != MessageBoxResult.Yes)
{
return;
}

try
{
string query;
NpgsqlParameter[] parameters;

query = @"INSERT INTO person(first_name, last_name,
patronymic, login, password, rights)
VALUES(:first_name, :last_name, :patronymic, :login, :password,
:rights::user_role)";

parameters = new NpgsqlParameter[]
{
new NpgsqlParameter(":first_name", name),
new NpgsqlParameter(":last_name", lastName),
new NpgsqlParameter(":patronymic",
string.IsNullOrEmpty(patronymic) ? (object)DBNull.Value :
patronymic),
new NpgsqlParameter(":login", login),
new NpgsqlParameter(":password", password),
new NpgsqlParameter(":rights", types)
};

db.ExecuteNonQuery(query, parameters);

string getUserIdQuery = @"
SELECT person_id
FROM person
WHERE first_name = :first_name
AND last_name = :last_name
AND (patronymic = :patronymic OR (patronymic IS NULL AND
:patronymic IS NULL))
AND login = :login";

var userIdParams = new NpgsqlParameter[]
{
new NpgsqlParameter(":first_name", name),
new NpgsqlParameter(":last_name", lastName),
new NpgsqlParameter(":patronymic",
string.IsNullOrEmpty(patronymic) ? (object)DBNull.Value :
patronymic),
new NpgsqlParameter(":login", login)
};

var userIdResult = db.ExecuteQuery(getUserIdQuery,
userIdParams);

if (userIdResult == null || userIdResult.Count == 0)
{
MessageBox.Show("Ошибка получения ID пользователя");
return;
}

int personId = Convert.ToInt32(userIdResult[0]["person_id"]);

if (types == "Студент" && Group.SelectedItem != null)
{
string groupName = Group.SelectedItem.ToString();
string getGroupIdQuery = "SELECT group_id FROM
student_group WHERE group_name = @groupName";
var groupIdResult = db.ExecuteQuery(getGroupIdQuery,
new NpgsqlParameter("@groupName", groupName));

if (groupIdResult == null || groupIdResult.Count == 0)
{
MessageBox.Show("Группа не найдена в базе данных!");
return;
}

int groupId = Convert.ToInt32(groupIdResult[0]["group_id"]);

string updateStudentQuery = @"
UPDATE person
SET fk_group_id = :group_id
WHERE person_id = :person_id";

var updateParams = new NpgsqlParameter[]
{
new NpgsqlParameter(":group_id", groupId),
new NpgsqlParameter(":person_id", personId)
};

db.ExecuteNonQuery(updateStudentQuery, updateParams);
}

if (types == "Преподаватель" && subjects.Count > 0)
{
foreach (string subject in subjects)
{
string getSubjectIdQuery = "SELECT subject_id FROM subject
WHERE title = @subject";
var subjectIdResult = db.ExecuteQuery(getSubjectIdQuery,
new NpgsqlParameter("@subject", subject));

if (subjectIdResult == null || subjectIdResult.Count == 0)
{
MessageBox.Show($"Предмет '{subject}' не найден в базе
данных");
continue;
}

int subjectId = Convert.ToInt32(subjectIdResult[0]["subject_id"]);

string subjectProfessorQuery = @"
INSERT INTO subject_professor (subject_id, professor_id)
VALUES (@subject_id, @professor_id)
ON CONFLICT (subject_id, professor_id) DO NOTHING";

var subjectProfessorParams = new NpgsqlParameter[]
{
new NpgsqlParameter("@subject_id", subjectId),
new NpgsqlParameter("@professor_id", personId)
};

try
{
```

```
db.ExecuteNonQuery(subjectProfessorQuery,
subjectProfessorParams);
}
catch (Exception ex)
{
MessageBox.Show($"Ошибка при добавлении дисциплины
'{subject}': {ex.Message}");
}
}
}

ClearAll();
MessageBox.Show("Учетная запись успешно создана!");
}
catch (Exception ex)
{
MessageBox.Show($"Ошибка при создании учетной записи:
{ex.Message}\n{ex.StackTrace}");
}
}

private bool IsPersonExists(string firstName, string lastName,
string patronymic)
{
try
{
string query = @"
SELECT COUNT(*)
FROM person
WHERE first_name = :first_name
  AND last_name = :last_name
  AND (patronymic = :patronymic OR (patronymic IS NULL AND
:patronymic IS NULL))";
var parameters = new NpgsqlParameter[]
{
new NpgsqlParameter(":first_name", firstName),
new NpgsqlParameter(":last_name", lastName),
new NpgsqlParameter(":patronymic",
string.IsNullOrEmpty(patronymic) ? (object)DBNull.Value :
patronymic)
};
var result = db.ExecuteQuery(query, parameters);
if (result != null && result.Count > 0 && result[0][0] != null)
{
return Convert.ToInt32(result[0][0]) > 0;
}
return false;
}
catch (Exception ex)
{
MessageBox.Show($"Ошибка проверки пользователя:
{ex.Message}");
return false;
}
}

private bool CheckError()
{
Error.Visibility = Visibility.Collapsed;

if (string.IsNullOrWhiteSpace(Name.Text) ||
string.IsNullOrWhiteSpace(LastName.Text) ||
string.IsNullOrWhiteSpace(Login.Text) ||
string.IsNullOrWhiteSpace(Password.Text) ||
Types.SelectedItem == null)
{
Error.Text = "Заполните все обязательные поля!";
Error.Visibility = Visibility.Visible;
return true;
}
if (Types.SelectedItem.ToString() == "Студент")
{
if (Group.SelectedItem == null)
{
Error.Text = "Для студента необходимо выбрать группу!";
Error.Visibility = Visibility.Visible;
return true;
}
}

if (Login.Text.Trim().Length < 5)
{
Error.Text = "Логин должен быть не менее 5 символов!";
Error.Visibility = Visibility.Visible;
return true;
}

if (Password.Text.Trim().Length < 8)
{
Error.Text = "Пароль должен быть не менее 8 символов!";
Error.Visibility = Visibility.Visible;
return true;
}

if (check_Space(Login.Text) || check_Space(Password.Text))
{
Error.Text = "Логин и пароль не должны содержать пробелы!";
Error.Visibility = Visibility.Visible;
return true;
}

if (check_SpecialChars(Login.Text))
{
Error.Text = "Логин содержит недопустимые символы!";
Error.Visibility = Visibility.Visible;
return true;
}

if (Types.SelectedItem.ToString() == "Преподаватель")
{
if (Subject.SelectedItem != null)
{
string selectedSubject = Subject.SelectedItem.ToString();
if (!subjects.Contains(selectedSubject))
{
subjects.Add(selectedSubject);
}
}
if (subjects.Count == 0)
{
Error.Text = "Выберите хотя бы одну дисциплину для
преподавателя!";
Error.Visibility = Visibility.Visible;
return true;
}
}
try
{
string query = @"SELECT COUNT(*) FROM person WHERE
login = :login";
var parameter = new NpgsqlParameter(":login",
Login.Text.Trim());
var result = db.ExecuteQuery(query, parameter);
if (result != null && result.Count > 0 && result[0][0] != null)
{
int count = Convert.ToInt32(result[0][0]);
if (count > 0)
{
Error.Text = "Такой логин уже существует!";
Error.Visibility = Visibility.Visible;
return true;
}
}
}
catch (Exception ex)
{
MessageBox.Show($"Ошибка проверки логина: {ex.Message}");
return true;
}
return false;
}

public bool check_Space(string input) => input.Any(ch =>
char.IsWhiteSpace(ch));
public bool check_SpecialChars(string input) =>
input.Any(ch => !char.IsLetterOrDigit(ch) && ch != '_' && ch !=
'-');
private void ClearAll()
```

```
{
Name.Text = string.Empty;
Login.Text = string.Empty;
Password.Text = string.Empty;
Types.SelectedItem = null;
LastName.Text = string.Empty;
Patronymic.Text = string.Empty;
Subject.SelectedItem = null;
Group.SelectedItem = null;
subjects.Clear();
SubText.Visibility = Visibility.Collapsed;
Subject.Visibility = Visibility.Collapsed;
newSub.Visibility = Visibility.Collapsed;
Add.Visibility = Visibility.Collapsed;
GroupText.Visibility = Visibility.Collapsed;
Group.Visibility = Visibility.Collapsed;
Error.Visibility = Visibility.Collapsed;
}
private void Page_Loaded(object sender, RoutedEventArgs e)
{
ReturnButton.Visibility = SaveData.isNewProfile ?
Visibility.Visible : Visibility.Collapsed;
try
{
var query = @"SELECT DISTINCT title FROM subject ORDER
BY title";
var subjectDB = db.ExecuteQuery(query);
if (subjectDB != null)
{
var subject = subjectDB.Cast<DataRowView>()
.Select(row => row["title"].ToString())
.ToList();
Subject.ItemsSource = subject;
}
}
catch (Exception ex)
{
MessageBox.Show($"Ошибка загрузки предметов:
{ex.Message}");
}
try
{
var query = @"SELECT group_name FROM student_group
ORDER BY group_name";
var groupDB = db.ExecuteQuery(query);
if (groupDB != null)
{
var groups = groupDB.Cast<DataRowView>()
.Select(row => row["group_name"].ToString())
.ToList();
Group.ItemsSource = groups;}}
catch (Exception ex)
{
MessageBox.Show($"Ошибка загрузки групп: {ex.Message}");
}
List<string> types = new List<string>()
{
"Администратор",
"Студент",
"Преподаватель"};
Types.ItemsSource = types;
subjects.Clear();}
private void newSub_Click(object sender, RoutedEventArgs e){
SaveData.isNewProfile = true;
((MainForm)Window.GetWindow(this)).OpenNewSub();}
private void ReturnButton_Click(object sender, RoutedEventArgs
e){
SaveData.isNewProfile = false;
((MainForm)Window.GetWindow(this)).OpenNewSub();}
private void Types_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
if (Types.SelectedItem != null)
{
string selectedType = Types.SelectedItem.ToString();
bool isTeacher = selectedType == "Преподаватель";
bool isStudent = selectedType == "Студент";
SubText.Visibility = isTeacher ? Visibility.Visible :
Visibility.Collapsed;
Subject.Visibility = isTeacher ? Visibility.Visible :
Visibility.Collapsed;
newSub.Visibility = isTeacher ? Visibility.Visible :
Visibility.Collapsed;
Add.Visibility = isTeacher ? Visibility.Visible :
Visibility.Collapsed;
GroupText.Visibility = isStudent ? Visibility.Visible :
Visibility.Collapsed;
Group.Visibility = isStudent ? Visibility.Visible :
Visibility.Collapsed;
if (!isTeacher)
{
subjects.Clear();
}
}
}
private void Subject_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
if (Subject.SelectedItem != null &&
Types.SelectedItem?.ToString() == "Преподаватель")
{
string selectedSubject = Subject.SelectedItem.ToString();
if (!subjects.Contains(selectedSubject))
{
subjects.Add(selectedSubject);
}
}
}
private void Name_PreviewTextInput(object sender,
System.Windows.Input.TextCompositionEventArgs e) =>
e.Handled = !Regex.IsMatch(e.Text, @"^[a-zA-Za-яA-Я]+$");
private void Login_PreviewTextInput(object sender,
System.Windows.Input.TextCompositionEventArgs e) =>
e.Handled = !Regex.IsMatch(e.Text, @"^[a-zA-Z0-9_-]+$");
private void LastName_PreviewTextInput(object sender,
System.Windows.Input.TextCompositionEventArgs e) =>
e.Handled = !Regex.IsMatch(e.Text, @"^[a-zA-Za-яA-Я]+$");
private void Patronymic_PreviewTextInput(object sender,
System.Windows.Input.TextCompositionEventArgs e) =>
e.Handled = !Regex.IsMatch(e.Text, @"^[a-zA-Za-яA-Я]+$");
private void Password_PreviewTextInput(object sender,
System.Windows.Input.TextCompositionEventArgs e)
{
e.Handled = false;
}
private void Name_LostFocus(object sender, RoutedEventArgs e)
{
if (!string.IsNullOrWhiteSpace(Name.Text) &&
check_SpecialChars(Name.Text))
{
MessageBox.Show("Имя может содержать только буквы!");
Name.Text = string.Empty;
}
}
private void Login_LostFocus(object sender, RoutedEventArgs e)
{
if (!string.IsNullOrWhiteSpace(Login.Text) &&
check_Space(Login.Text))
{
MessageBox.Show("Логин не должен содержать пробелы!");
Login.Text = Login.Text.Replace(" ", "");
}
}
private void Add_Click(object sender, RoutedEventArgs e)
{
if (Subject.SelectedItem != null)
{
string selectedSubject = Subject.SelectedItem.ToString();
if (!subjects.Contains(selectedSubject))
{
subjects.Add(selectedSubject);
MessageBox.Show($"Дисциплина '{selectedSubject}' добавлена
в список");}
else{
MessageBox.Show($"Дисциплина '{selectedSubject}' уже
добавлена");}}
else{
MessageBox.Show("Выберите дисциплину из списка");}}}}
```

```
using Npgsql;
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text.RegularExpressions;
using System.Windows;
using System.Windows.Controls;
namespace WpfApp2
{
public partial class newSub : Page
{
public newSub()
{
InitializeComponent();
}
database db = new database();
List<string> profs = new List<string>();
List<string> selectedGroups = new List<string>();
private void Save_Click(object sender, RoutedEventArgs e)
{
if (CheckError())
{
return;
}
bool isOffset = IsOffset.IsChecked ?? false;
string title = Name.Text.Trim();
string listProf = string.Join("\n", profs);
string listGroups = string.Join("\n", selectedGroups);
string act1 = SaveData.isChange ? "изменить" : "создать";
string action = SaveData.isChange ? "изменен" : "создан";
MessageBoxResult result = MessageBox.Show(
$"{char.ToUpper(act1[0])}{act1.Substring(1)} дисциплину
{title}\n" +
$"Тип: {(isOffset ? "Зачет" : "Экзамен")}\n" +
$"Преподаватели:\n{listProf}\n" +
$"Группы:\n{listGroups}",
"Подтверждение",
MessageBoxButton.YesNo,
MessageBoxImage.Question);
if (result == MessageBoxResult.Yes)
{
try
{
if (SaveData.isChange)
{
string getSubjectIdQuery = "SELECT subject_id FROM subject
WHERE title = @currentTitle";
var subjectIdResult = db.ExecuteQuery(getSubjectIdQuery,
new NpgsqlParameter("@currentTitle", SaveData.currentSub));
if (subjectIdResult != null && subjectIdResult.Count > 0)
{
int subject = Convert.ToInt32(subjectIdResult[0]["subject_id"]);
string updateSubjectQuery = $@"
UPDATE subject
SET title = @title,
isoffset = {isOffset}
WHERE subject_id = @subjectId";
db.ExecuteNonQuery(updateSubjectQuery, new[]
{
new NpgsqlParameter("@title", title),
new NpgsqlParameter("@subjectId", subject)
});
string deleteSubjectProfessorQuery =
"DELETE FROM subject_professor WHERE subject_id =
@subjectId";
db.ExecuteNonQuery(deleteSubjectProfessorQuery,
new NpgsqlParameter("@subjectId", subject));
string deleteSubjectGroupQuery =
"DELETE FROM subject_group WHERE subject_id = @subjectId";
db.ExecuteNonQuery(deleteSubjectGroupQuery,
new NpgsqlParameter("@subjectId", subject));
}
}
else
{
string insertSubjectQuery = @"
INSERT INTO subject (title, isoffset)
VALUES (@title, @isoffset)
RETURNING subject_id";
  var subjectIdResult = db.ExecuteQuery(insertSubjectQuery, new[]
{
new NpgsqlParameter("@title", title),
new NpgsqlParameter("@isoffset", isOffset)
});
if (subjectIdResult == null || subjectIdResult.Count == 0)
{
MessageBox.Show("Ошибка создания предмета!");
return;
}
}
  string getSubjectIdForRelationsQuery = "SELECT subject_id
FROM subject WHERE title = @title";
var subjectIdForRelationsResult = db.ExecuteQuery(
getSubjectIdForRelationsQuery,
new NpgsqlParameter("@title", title));

if (subjectIdForRelationsResult == null ||
subjectIdForRelationsResult.Count == 0)
{
MessageBox.Show($"Предмет {title} не найден!");
return;
}

int subjectId =
Convert.ToInt32(subjectIdForRelationsResult[0]["subject_id"]);
foreach (string groupName in selectedGroups)
{
string getGroupIdQuery = "SELECT group_id FROM student_group
WHERE group_name = @groupName";
var groupIdResult = db.ExecuteQuery(getGroupIdQuery,
new NpgsqlParameter("@groupName", groupName));

if (groupIdResult == null || groupIdResult.Count == 0)
{
MessageBox.Show($"Группа '{groupName}' не найдена!");
continue;
}

int groupId = Convert.ToInt32(groupIdResult[0]["group_id"]);

string insertSubjectGroupQuery = @"
INSERT INTO subject_group (subject_id, group_id)
VALUES (@subjectId, @groupId)
ON CONFLICT (subject_id, group_id) DO NOTHING";
db.ExecuteNonQuery(insertSubjectGroupQuery, new[]
{
new NpgsqlParameter("@subjectId", subjectId),
new NpgsqlParameter("@groupId", groupId)
});
}
foreach (string prof in profs)
{
var parts = ParseFIO(prof);
if (parts == null) continue;
string lastName = parts[0];
string firstName = parts[1];
string patronymic = parts[2];
string getProfessorIdQuery = @"
SELECT person_id
FROM person
WHERE first_name = @firstName
  AND last_name = @lastName
```

```
  AND (patronymic = @patronymic OR (patronymic IS NULL AND
@patronymic IS NULL))";
var professorParams = new[]
{
new NpgsqlParameter("@firstName", firstName),
new NpgsqlParameter("@lastName", lastName),
new NpgsqlParameter("@patronymic",
string.IsNullOrEmpty(patronymic) ?
(object)DBNull.Value : patronymic)
};
var professorIdResult = db.ExecuteQuery(getProfessorIdQuery,
professorParams);
if (professorIdResult == null || professorIdResult.Count == 0)
{
MessageBox.Show($"Преподаватель {prof} не найден!");
continue;
}
int professorId =
Convert.ToInt32(professorIdResult[0]["person_id"]);

string insertSubjectProfessorQuery = @"
INSERT INTO subject_professor (subject_id, professor_id)
VALUES (@subjectId, @professorId)
ON CONFLICT (subject_id, professor_id) DO NOTHING";
db.ExecuteNonQuery(insertSubjectProfessorQuery, new[]
{
new NpgsqlParameter("@subjectId", subjectId),
new NpgsqlParameter("@professorId", professorId)
});
}
MessageBox.Show($"Дисциплина успешно {action}а!");
ClearAll();
}
catch (Exception ex)
{
MessageBox.Show($"Ошибка: {ex.Message}\n{ex.StackTrace}");
}
}
}
private string[] ParseFIO(string fio)
{
if (string.IsNullOrWhiteSpace(fio))
return null;
var parts = fio.Trim().Split(new[] { ' ' },
StringSplitOptions.RemoveEmptyEntries)
if (parts.Length < 2)
return null;
string lastName = parts[0];
string firstName = parts[1];
string patronymic = parts.Length > 2 ? parts[2] : null;
return new[] { lastName, firstName, patronymic };
}
private bool CheckError()
{
Error.Visibility = Visibility.Collapsed;
if (string.IsNullOrWhiteSpace(Name.Text))
{
Error.Text = "Введите название дисциплины!";
Error.Visibility = Visibility.Visible;
return true;
}
if (selectedGroups.Count == 0)
{
Error.Text = "Выберите хотя бы одну группу!";
Error.Visibility = Visibility.Visible;
return true;
}
string title = Name.Text.Trim();
if (title.Length < 3 || title.Length > 100)
{
Error.Text = "Название дисциплины должно быть от 3 до 100
символов!";
Error.Visibility = Visibility.Visible;
return true;
}
if (ContainsSpecialChars(title))
{
Error.Text = "Название содержит недопустимые символы!";
Error.Visibility = Visibility.Visible;
return true;
}
if (profs.Count == 0)
{
Error.Text = "Выберите хотя бы одного преподавателя!";
Error.Visibility = Visibility.Visible;
return true;
}
foreach (string prof in profs)
{
var parts = ParseFIO(prof);
if (parts == null || parts.Length < 2)
{
Error.Text = "Некорректное ФИО преподавателя!";
Error.Visibility = Visibility.Visible;
return true;
}
}
if (profs.Count != profs.Distinct().Count())
{
Error.Text = "Преподаватель выбран более одного раза!";
Error.Visibility = Visibility.Visible;
return true;
}
if (selectedGroups.Count != selectedGroups.Distinct().Count())
{
Error.Text = "Группа выбрана более одного раза!";
Error.Visibility = Visibility.Visible;
return true;
}
if (IsProfessorSubjectDuplicate(profs, title))
{
Error.Text = "Один или несколько преподавателей уже ведут эту
дисциплину!";
Error.Visibility = Visibility.Visible;
return true;
}
if (!SaveData.isChange)
{
if (IsSubjectExists(title))
{
Error.Text = "Дисциплина с таким названием уже существует!";
Error.Visibility = Visibility.Visible;
return true;
}
}
return false;
}
private bool IsProfessorSubjectDuplicate(List<string> professors,
string subjectTitle)
{
try
{
foreach (string prof in professors)
{
var parts = ParseFIO(prof);
if (parts == null) continue;
string lastName = parts[0];
string firstName = parts[1];
string patronymic = parts[2];
string query = @"
SELECT COUNT(*)
FROM subject_professor sp
JOIN subject s ON sp.subject_id = s.subject_id
JOIN person p ON sp.professor_id = p.person_id
WHERE s.title = @subjectTitle
  AND p.last_name = @lastName
```

```
  AND p.first_name = @firstName
  AND (p.patronymic = @patronymic OR (p.patronymic IS NULL
AND @patronymic IS NULL))";
var parameters = new NpgsqlParameter[]
{
new NpgsqlParameter("@subjectTitle", subjectTitle),
new NpgsqlParameter("@lastName", lastName),
new NpgsqlParameter("@firstName", firstName),
new NpgsqlParameter("@patronymic",
string.IsNullOrEmpty(patronymic) ?
(object)DBNull.Value : patronymic)
};
var result = db.ExecuteQuery(query, parameters);
if (result != null && result.Count > 0 && result[0][0] != null)
{
int count = Convert.ToInt32(result[0][0]);
if (count > 0)
{
return true;
}
}
}
return false;
}
catch (Exception ex)
{
MessageBox.Show($"Ошибка проверки дублирования:
{ex.Message}");
return true;
}
}
private bool ContainsSpecialChars(string input)
{
return Regex.IsMatch(input, @"[^\p{L}\p{N}\s\-\(\)\,\.]");
}
private bool IsSubjectExists(string title)
{
try
{
string query = @"SELECT COUNT(*) FROM subject WHERE title
= :title";
var parameter = new NpgsqlParameter("title", title);
var result = db.ExecuteQuery(query, parameter);
if (result != null && result.Count > 0 && result[0][0] != null)
{
int count = Convert.ToInt32(result[0][0]);
return count > 0;
}
return false;
}
catch (Exception ex)
{
MessageBox.Show($"Ошибка проверки дисциплины:
{ex.Message}");
return true;
}
}
private void ClearAll()
{
Profs.SelectedItem = null;
Groups.SelectedItem = null;
Name.Text = string.Empty;
IsOffset.IsChecked = false;
profs.Clear();
selectedGroups.Clear();
Error.Visibility = Visibility.Collapsed;
}
private void Page_Loaded(object sender, RoutedEventArgs e)
{
profs.Clear();
selectedGroups.Clear();
if (SaveData.isChange)
{
NewSabText.Text = "Изменить дисциплину";
Save.Content = "Изменить";
Name.Text = SaveData.currentSub;
LoadCurrentProfessors();
LoadCurrentGroups();
try
{
string getSubjectQuery = "SELECT isoffset FROM subject WHERE
title = @currentSub";
var subjectResult = db.ExecuteQuery(getSubjectQuery,
new NpgsqlParameter("@currentSub", SaveData.currentSub));
if (subjectResult != null && subjectResult.Count > 0)
{
bool currentIsOffset =
Convert.ToBoolean(subjectResult[0]["isoffset"]);
IsOffset.IsChecked = currentIsOffset;
}
}
catch (Exception ex)
{
MessageBox.Show($"Ошибка загрузки параметров предмета:
{ex.Message}");
}
}
ReturnButton.Visibility = SaveData.isNewProfile ? Visibility.Visible
: Visibility.Collapsed;
var query = @"SELECT TRIM(CONCAT(p.last_name, ' ',
p.first_name, ' ', COALESCE(p.patronymic, ' '))) AS ""prof""
FROM person p
WHERE rights='Преподаватель'";
var profsDB = db.ExecuteQuery(query);
if (profsDB != null)
{
var profsList = profsDB.Cast<DataRowView>().Select(row =>
row["prof"].ToString()).ToList();
Profs.ItemsSource = profsList;
}

List<string> groups = new List<string>();
string groupsQuery = "SELECT group_name FROM student_group
ORDER BY group_name";
var groupsResult = db.ExecuteQuery(groupsQuery);
if (groupsResult != null)
{
foreach (DataRowView row in groupsResult)
{
groups.Add(row["group_name"].ToString());
}
}
if (groups.Count == 0)
{
groups.Add("Информационные технологии и
программирование");
}
Groups.ItemsSource = groups;
}
private void LoadCurrentProfessors()
{
try
{
string query = @"
SELECT TRIM(CONCAT(p.last_name, ' ', p.first_name, ' ',
COALESCE(p.patronymic, ' '))) AS prof
FROM subject_professor sp
JOIN subject s ON sp.subject_id = s.subject_id
JOIN person p ON sp.professor_id = p.person_id
WHERE s.title = @currentSub";
var result = db.ExecuteQuery(query,
new NpgsqlParameter("@currentSub", SaveData.currentSub));
if (result != null)
{
profs.Clear();
foreach (DataRowView row in result)
```

```
{
string prof = row["prof"].ToString().Trim();
if (!string.IsNullOrEmpty(prof) && !profs.Contains(prof))
{
profs.Add(prof);
}
}
}
}
catch (Exception ex)
{
MessageBox.Show($"Ошибка загрузки преподавателей:
{ex.Message}");
}
}
private void LoadCurrentGroups()
{
try
{
string query = @"
SELECT g.group_name
FROM subject_group sg
JOIN subject s ON sg.subject_id = s.subject_id
JOIN student_group g ON sg.group_id = g.group_id
WHERE s.title = @currentSub";
var result = db.ExecuteQuery(query,
new NpgsqlParameter("@currentSub", SaveData.currentSub));
if (result != null)
{
selectedGroups.Clear();
foreach (DataRowView row in result)
{
string group = row["group_name"].ToString().Trim();
if (!string.IsNullOrEmpty(group) &&
!selectedGroups.Contains(group))
{
selectedGroups.Add(group);
}
}
}}
catch (Exception ex)
{
MessageBox.Show($"Ошибка загрузки групп предмета:
{ex.Message}");
}
}
private void newProf_Click(object sender, RoutedEventArgs e)
{
SaveData.isNewProfile = true;
((MainForm)Window.GetWindow(this)).OpenNewProf();
}
private void ReturnButton_Click(object sender, RoutedEventArgs e)
{
SaveData.isNewProfile = false;
((MainForm)Window.GetWindow(this)).OpenNewProf();
}
private void Profs_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
if (Profs.SelectedItem != null)
{
string selectedProf = Profs.SelectedItem.ToString();
if (!profs.Contains(selectedProf))
{
profs.Add(selectedProf);
}}}
private void Groups_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
if (Groups.SelectedItem != null)
{
string selectedGroup = Groups.SelectedItem.ToString();
if (!selectedGroups.Contains(selectedGroup))
{
selectedGroups.Add(selectedGroup);
}
}}
private void Add_Click(object sender, RoutedEventArgs e)
{
if (Profs.SelectedItem != null)
{
string selectedProf = Profs.SelectedItem.ToString();
if (!profs.Contains(selectedProf))
{
profs.Add(selectedProf);
MessageBox.Show($"Преподаватель '{selectedProf}' добавлен в
список");
}
else
{
MessageBox.Show($"Преподаватель '{selectedProf}' уже в
списке");
}
}
else
{
MessageBox.Show("Выберите преподавателя");
}
}
private void AddGroup_Click(object sender, RoutedEventArgs e)
{
if (Groups.SelectedItem != null)
{
string selectedGroup = Groups.SelectedItem.ToString();
if (!selectedGroups.Contains(selectedGroup))
{
selectedGroups.Add(selectedGroup);
MessageBox.Show($"Группа '{selectedGroup}' добавлена в
список");
}
else
{
MessageBox.Show($"Группа '{selectedGroup}' уже в списке");
}
}
else
{
MessageBox.Show("Выберите группу");
}}
private void Name_TextChanged(object sender,
TextChangedEventArgs e)
{
if (Name.Text.Length > 100)
{
Error.Text = "Название не должно превышать 100 символов!";
Error.Visibility = Visibility.Visible;
}
else if (Error.Visibility == Visibility.Visible && Name.Text.Length
>= 3)
{
Error.Visibility = Visibility.Collapsed;
}}
private void Name_PreviewTextInput(object sender,
System.Windows.Input.TextCompositionEventArgs e)
{
e.Handled = !Regex.IsMatch(e.Text, @"^[\p{L}\p{N}\s\-\(\)\,\.]+$");
}}}
```

ButtonStyleExtensions.cs

```
using System.Windows;
using System.Windows.Media;
namespace WpfApp2
{
public static class ButtonStyleExtensions
{
public static readonly DependencyProperty NormalColorProperty =
DependencyProperty.RegisterAttached(
"NormalColor",
typeof(Brush),
typeof(ButtonStyleExtensions),
new FrameworkPropertyMetadata(new
SolidColorBrush(Color.FromRgb(239, 165, 80))));

public static void SetNormalColor(DependencyObject element,
Brush value) =>
element.SetValue(NormalColorProperty, value);

public static Brush GetNormalColor(DependencyObject element) =>
(Brush)element.GetValue(NormalColorProperty);

public static readonly DependencyProperty HoverColorProperty =
DependencyProperty.RegisterAttached(
"HoverColor",
typeof(Brush),
typeof(ButtonStyleExtensions),
new FrameworkPropertyMetadata(new
SolidColorBrush(Color.FromRgb(255, 187, 110))));
public static void SetHoverColor(DependencyObject element, Brush
value) =>
element.SetValue(HoverColorProperty, value);
public static Brush GetHoverColor(DependencyObject element) =>
(Brush)element.GetValue(HoverColorProperty);
public static readonly DependencyProperty PressedColorProperty =
DependencyProperty.RegisterAttached(
"PressedColor",
typeof(Brush),
typeof(ButtonStyleExtensions),
new FrameworkPropertyMetadata(new
SolidColorBrush(Color.FromRgb(255, 168, 69))));
public static void SetPressedColor(DependencyObject element,
Brush value) =>
element.SetValue(PressedColorProperty, value);

public static Brush GetPressedColor(DependencyObject element) =>
(Brush)element.GetValue(PressedColorProperty);
}
}
```

SaveRole.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace WpfApp2{
public static class SaveData{
public static string role;
public static string name;
public static string id;
public static List<string> subjects { get; set; } = new List<string>();
public static string login;
public static string password;
public static string currentSub;
public static double currentWidth;

public static bool isChange = false;
public static bool isNewProfile;
}
```

Database.cs

```
using Npgsql;
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.SqlClient;
using System.Linq;
using System.Security.Cryptography;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Markup;
using System.Windows.Media;
namespace WpfApp2{
internal class database{
string connectionString = "host=localhost;port =
5433;database=grade;username = postgres;password
=123;sslmode=prefer;timeout = 10";
public DataView ExecuteQuery(string query, params
NpgsqlParameter[] parameters){
using (var connection = new NpgsqlConnection(connectionString))
using (var command = new NpgsqlCommand(query, connection)){
try{
foreach(var param in parameters){
command.Parameters.Add(param);}
connection.Open();
DataTable dataTable = new DataTable();
dataTable.Load(command.ExecuteReader());
return dataTable.DefaultView;}
catch (Exception ex){
MessageBox.Show($"{ex.Message}");
return null;}
}}
public DataTable DataQuery(string query, params NpgsqlParameter[]
parameters)
{
DataTable dt = new DataTable();
try
{
using (var connection = new NpgsqlConnection(connectionString))
{
connection.Open();
using (var command = new NpgsqlCommand(query, connection))
{
foreach (var param in parameters)
{
command.Parameters.Add(param);
}
using (var adapter = new NpgsqlDataAdapter(command))
adapter.Fill(dt);
}}}
catch (Exception ex){
MessageBox.Show($"{ex.Message}");}
return dt;}
public void ExecuteNonQuery(string query, params
NpgsqlParameter[] parameters){
try
{
using (var connection = new NpgsqlConnection(connectionString))
using (var command = new NpgsqlCommand(query, connection))
{
connection.Open();
if (parameters != null && parameters.Length > 0)
command.Parameters.AddRange(parameters);
int changed = command.ExecuteNonQuery();
}}
catch (Exception ex)
{
MessageBox.Show($"Ошибка сохранения: {ex.Message}");
}}}
}
```